

RV32I Single-cycle SoC & Mini OS 상세 매뉴얼

교육용 단일 사이클 RV32I 프로세서 · SoC · 펌웨어 toolchain · 빌드/시뮬레이션 스크립트 · Mini OS

프로젝트	RISC-V/ (rtl, firmware, scripts, tb, constraints, docs)
대상 보드	Digilent Nexys A7-100T (xc7a100tcsg324-1), Vivado 2022.1
Toolchain	riscv64-unknown-elf-gcc 9.3.0 (-march=rv32i -mabi=ilp32), Icarus Verilog
생성일	2026-09-27
원본	docs/MANUAL.md (scripts/build_manual_pdf.py로 생성)

이 문서는 RISC-V/ 저장소에 있는 교육용 RV32I 프로세서, SoC, 펌웨어 toolchain, 빌드/시뮬레이션 스크립트, 그리고 그 위에 올릴 mini OS를 기본 개념부터 차근차근 설명합니다.

모든 설명은 저장소의 실제 코드(rtl/ , firmware/ , scripts/ , tb/ , constraints/ , Makefile)와 실제 빌드 산출물(build/)을 근거로 합니다.

목차

1. 이 문서를 읽는 방법과 저장소 구조	4
1.1 대상 독자와 읽는 순서	4
1.2 저장소 구조	4
1.3 이 저장소가 "하는 일" 한 문장 요약	5
2. 기본 개념	6
2.1 프로세서는 무엇을 하는가	6
2.2 RISC-V ISA와 RV32I	6
2.3 레지스터와 ABI 이름	7
2.4 명령어 형식(R/I/S/B/U/J)과 immediate	7
2.5 Single-cycle, multi-cycle, pipeline	8
2.6 메모리, little-endian, 정렬, MMIO	8
2.7 Privileged 개념: M-mode, CSR, trap, ECALL/MRET	9
2.8 이 프로젝트를 읽는 데 필요한 Verilog 최소 지식	10
3. 하드웨어 전체 구조	11
3.1 모듈 계층	11
3.2 메모리 맵	11
3.3 두 개의 clock domain (FPGA)	12
3.4 코어와 SoC 사이의 인터페이스 계약	12
4. RTL 상세 설명	13
4.1 rtl/rv32_alu.v — 조합식 산술/논리 연산기	13
4.2 rtl/rv32_regfile.v — 2R1W 레지스터 파일	14
4.3 rtl/rv32_csr.v — M-mode CSR와 trap 상태	14
4.4 rtl/rv32_core.v — 단일 사이클 RV32I 코어	16
4.5 rtl/simple_timer.v — machine timer	21
4.6 rtl/rv32_soc.v — 메모리와 MMIO를 묶은 SoC	22
4.7 rtl/uart_tx.v — 8-N-1 UART 송신기	24
4.8 rtl/nexys_a7_top.v — 보드 top	25
4.9 한 명령어가 한 사이클에 흐르는 경로 추적	26
5. 펌웨어(소프트웨어) 상세 설명	28
5.1 개념: 소스 코드가 메모리 초기값이 되기까지	28
5.2 firmware/linker.ld — 링커 스크립트	28
5.3 firmware/nexys_hello.S — 어셈블리 데모 (FPGA 기본 이미지)	30
5.4 firmware/c_demo.c — freestanding C 데모	31
5.5 HEX 파일 형식과 scripts/bin2hex.py	34
6. 빌드 시스템과 스크립트 상세 설명	35
6.1 Makefile 읽는 법 (기초)	35
6.2 변수 정의	35
6.3 펌웨어 타겟	36
6.4 C 데모 타겟	37
6.5 시뮬레이션·lint·FPGA·정리 타겟	38
6.6 명령어 요약 (무엇을 치면 무엇이 생기나)	38
6.7 scripts/run_c_demo_sim.sh	39
6.8 constraints/nexys_a7_100t.xdc	40

6.9 scripts/build_nexys_a7.tcl — Vivado 비프로젝트 흐름	40
6.10 scripts/program_nexys_a7.tcl — 보드 프로그래밍	41
6.11 scripts/run_xsim.tcl — Vivado 시뮬레이터용	41
6.12 start.sh / stop.sh — 슬라이드 서버	42
7. 시뮬레이션과 검증	44
7.1 검증의 총위	44
7.2 tb/tb_rv32_core.v — smoke test	44
7.3 tb/tb_c_demo.v — C 데모 end-to-end	45
7.4 실행과 기대 출력	46
7.5 파형(VCD) 보는 법	46
8. FPGA 구현 흐름과 결과 해석	47
8.1 전체 순서	47
8.2 실제 결과 (build/vivado/*.rpt)	47
8.3 리포트 파일	47
8.4 펌웨어를 바꿔 FPGA에 올리기	48
9. Mini OS: 개념, 하드웨어 기반, 구현	49
9.1 OS란 무엇인가 (이 프로젝트의 관점에서)	49
9.2 현재 하드웨어가 제공하는 것	49
9.3 하드웨어에 없는 것 (구현이 우회하거나 감수한 것)	50
9.4 Trap 처리 흐름을 cycle 단위로	50
9.5 구현 개요	51
9.6 코드 상세	51
9.7 빌드와 시뮬레이션 결과	55
9.8 FPGA에서 실행하기	56
9.9 다음 단계	56
10. 자주 하는 질문과 문제 해결	58
부록 A. RV32I 명령어 인코딩 표	59
부록 B. 구현된 CSR 표	61
부록 C. 손으로 명령어 디코딩하기	62
부록 D. 용어집	63

1. 이 문서를 읽는 방법과 저장소 구조

1.1 대상 독자와 읽는 순서

- 프로세서를 처음 배우는 독자: 2장(기본 개념)을 먼저 끝까지 읽고, 3장 → 4.9절(사이클 추적) → 4.1~4.8 순서로 읽으면 좋습니다.
- 디지털 설계 경험이 있는 독자: 3장 → 4장 → 9장 순서로 읽고, 필요한 개념만 2장에서 찾아보면 됩니다.
- 소프트웨어/OS에 관심이 있는 독자: 2.7절 → 4.3절(CSR) → 4.5절(timer) → 5장 → 9장 순서를 권합니다.
- 빌드/스크립트만 이해하려는 독자: 6장과 7장만 읽어도 독립적으로 이해할 수 있게 작성했습니다.

1.2 저장소 구조

```
RISC-V/
├── rtl/
│   ├── rv32_alu.v          # 합성 가능한 Verilog RTL
│   ├── rv32_regfile.v     # ALU (조합 논리)
│   ├── rv32_csr.v         # x0~x31 레지스터 파일 (2R1W)
│   ├── rv32_core.v        # M-mode CSR, trap 진입/복귀 상태
│   ├── simple_timer.v     # 단일 사이클 RV32I 코어 (fetch~write-back)
│   ├── rv32_soc.v         # mtime/mtimecmp timer, level IRQ
│   ├── uart_tx.v          # 코어 + 통합 메모리 + MMIO(UART, timer)
│   └── nexys_a7_top.v     # 8-N-1 UART 송신기
├── firmware/
│   ├── nexys_hello.S      # Nexys A7-100T 보드 top (clock 분주, reset, UART 연결)
│   ├── nexys_hello.hex   # CPU에서 실행되는 프로그램
│   ├── c_demo.c          # 어셈블리 "Hello Nexys A7!" (FPGA 기본 이미지)
│   ├── c_demo.hex        # 위 프로그램의 $readmemh 이미지
│   ├── boot.S            # freestanding C 데모 (1..10 합, UART "C OK\n")
│   ├── trap.S            # 위 프로그램의 $readmemh 이미지 (시뮬레이션용)
│   ├── kernel.c          # mini OS: reset 진입 (sp, .bss, mtvec, kernel_main)
│   ├── mini_os.hex       # mini OS: trap 진입/복귀, context switch
│   ├── linker.ld         # mini OS: timer, round-robin scheduler, ecall syscall, task
│   └── linker.ld         # mini OS FPGA 이미지 (10 ms tick)
├── tb/
│   ├── tb_rv32_core.v    # 링커 스크립트 (RAM 8 KiB, _start를 0번지에, 스택/bss 심볼)
│   ├── tb_c_demo.v      # 테스트벤치
│   └── tb_mini_os.v      # 기계어를 직접 메모리에 넣는 smoke test
├── scripts/
│   ├── bin2hex.py        # c_demo.hex end-to-end 검증 + VCD 생성
│   ├── run_c_demo_sim.sh # mini OS: timer IRQ, syscall, task 전환 검증 + VCD 생성
│   ├── run_mini_os_sim.sh # 빌드/시뮬레이션/FPGA 스크립트
│   ├── build_nexys_a7.tcl # raw binary -> $readmemh HEX
│   ├── program_nexys_a7.tcl # Icarus Verilog로 tb_c_demo 실행
│   └── run_xsim.tcl       # Icarus Verilog로 tb_mini_os 실행
├── constraints/
│   └── nexys_a7_100t.xdc # Vivado 비프로젝트 batch 합성/구현/bitstream
├── slides/
│   └── index.html         # Vivado Hardware Manager로 보드 프로그래밍
├── build/
│   ├── firmware/        # Vivado xsim batch용 ("run all; quit")
│   ├── vivado/          # .elf .bin .s .dis .map, mini_os_sim.hex
│   ├── c_demo.vcd       # .bit, .dcp, timing/utilization 리포트
│   └── mini_os.vcd       # c_demo 시뮬레이션 파형
├── Makefile              # mini OS 시뮬레이션 파형
├── start.sh / stop.sh   # 모든 빌드 진입점
└── README.md             # 슬라이드용 로컬 HTTP 서버 시작/종료
```

1.3 이 저장소가 "하는 일" 한 문장 요약

C 또는 어셈블리로 작성한 프로그램을 RISC-V 기계어로 바꾸고(5장, 6장), 그 기계어를 Verilog로 설계한 RV32I CPU의 메모리에 넣어(4.6절), 시뮬레이터로 동작을 검증한 뒤(7장), FPGA에 올려 UART로 문자를 출력한다(8장). 그 CPU의 timer interrupt와 ECALL/MRET 위에서 세 task를 round-robin으로 돌리는 mini OS가 실제로 실행되고 시뮬레이션으로 검증된다(9장).

2. 기본 개념

이 장은 코드를 읽기 전에 알아야 할 개념만 골라서 설명합니다. 이미 알고 있는 부분은 건너뛰어도 됩니다.

2.1 프로세서는 무엇을 하는가

프로세서(CPU)는 메모리에 들어 있는 명령어를 순서대로 읽어 실행하는 기계입니다. 각 명령어 실행은 관습적으로 다섯 단계로 나눠 설명합니다.

단계	영어	하는 일	이 프로젝트에서 담당하는 코드
1. 인출	Fetch	PC(program counter)가 가리키는 주소의 명령어를 메모리에서 읽는다	rv32_core.v 의 imem_addr = pc , rv32_soc.v 의 irdata
2. 해독	Decode	32-bit 명령어를 opcode/funct/레지스터 번호/immediate로 쪼개고 무엇을 할지 결정한다	rv32_core.v 의 always @* 안 case (opcode)
3. 실행	Execute	ALU로 덧셈/비교/시프트 등을 하거나 분기 조건을 판단한다	rv32_alu.v , rv32_core.v 의 분기 비교
4. 메모리 접근	Memory	load/store라면 데이터 메모리를 읽거나 쓴다	rv32_core.v 의 daddr_r/dwstrb_r , rv32_soc.v 의 mem[]
5. 결과 기록	Write-back	결과를 목적지 레지스터(rd)에 쓰고 PC를 다음 명령어로 옮긴다	rv32_regfile.v 의 write port, rv32_core.v 의 pc <= next_pc

PC는 "다음에 실행할 명령어의 주소"를 담는 특별한 레지스터입니다. RV32I 명령어는 모두 4바이트이므로 보통 PC + 4 가 다음 명령어이고, 분기(branch)/점프(jump)일 때만 다른 곳으로 이동합니다.

2.2 RISC-V ISA와 RV32I

ISA(Instruction Set Architecture) 는 "소프트웨어가 보는 프로세서의 약속"입니다. 어떤 레지스터가 있고, 어떤 명령어가 있고, 각 명령어가 몇 번째 비트에 무엇을 담는지 정의합니다. 같은 ISA를 구현하면 내부 회로가 달라도 같은 프로그램을 실행할 수 있습니다.

RISC-V는 공개 표준 ISA입니다. 이름의 각 부분이 의미하는 바:

- **RV32I**: 32-bit 정수(Integer) 기본 명령어 집합. 이 프로젝트가 구현한 것입니다. 약 40개의 명령어로 구성됩니다.
- **Zicsr**: CSR(Control and Status Register) 접근 명령(csrrw , csrrs , ...). 이 프로젝트는 이것도 구현했습니다.
- **M/A/F/D/C**: 곱셈·나눗셈, 원자적 연산, 단정도/배정도 부동소수점, 압축(16-bit) 명령. 이 프로젝트는 **구현하지 않았습니다**. 그래서 컴파일 시 -march=rv32i 로 컴파일러에게 "이 명령어만 써라"라고 지시합니다(6장).

RV32I의 핵심 설계 철학:

1. **load/store 아키텍처**: 메모리에 접근하는 명령어는 load(lw , lb , ...)와 store(sw , sb , ...)뿐입니다. 덧셈 같은 연산은 레지스터끼리만 합니다.
2. **고정 길이 32-bit 명령어**: 디코더가 단순해집니다. 이 프로젝트의 디코더가 always @* 블록 하나로 끝나는 이유입니다.
3. **소스 레지스터 필드 위치가 항상 같음**: rs1 은 항상 bit [19:15], rs2 는 항상 [24:20], rd 는 항상 [11:7]입니다. 그래서 디코딩이 끝나기도 전에 레지스터 파일을 읽기 시작할 수 있습니다(rv32_core.v 64행).

2.3 레지스터와 ABI 이름

RV32I에는 32-bit 범용 레지스터 32개($x_0 \sim x_{31}$)와 PC가 있습니다. x_0 는 항상 0이며 써도 무시됩니다. 이 성질 덕분에 `addi x1, x0, 5` (" $x_1 = 0 + 5$ ")처럼 상수 로드, `jal x0, target` ("반환 주소 버리고 점프") 같은 일을 별도 명령 없이 할 수 있습니다.

컴파일러와 어셈블러는 번호 대신 ABI(Application Binary Interface) 이름을 씁니다. 두 이름을 모두 알아야 `.s` (ABI 이름)와 `.dis` (-M numeric 으로 번호 표기) 파일을 대조할 수 있습니다.

번호	ABI 이름	용도	함수 호출 시 보존 여부
x_0	zero	상수 0	—
x_1	ra	반환 주소(return address)	호출자 저장
x_2	sp	스택 포인터	피호출자 저장
x_3	gp	전역 포인터(small data 접근용)	—
x_4	tp	스레드 포인터	—
$x_5 - x_7$	$t_0 - t_2$	임시	호출자 저장
x_8	s_0 / fp	저장 레지스터 / 프레임 포인터	피호출자 저장
x_9	s_1	저장 레지스터	피호출자 저장
$x_{10} - x_{11}$	$a_0 - a_1$	함수 인자 / 반환값	호출자 저장
$x_{12} - x_{17}$	$a_2 - a_7$	함수 인자	호출자 저장
$x_{18} - x_{27}$	$s_2 - s_{11}$	저장 레지스터	피호출자 저장
$x_{28} - x_{31}$	$t_3 - t_6$	임시	호출자 저장

예: `build/firmware/c_demo.s`의 `add a5,a5,a4`는 `.dis`에서 `add x15,x15,x14`로 보입니다.

`nexys_hello.S`처럼 OS 없이 혼자 도는 코드는 ABI 규약을 따를 의무가 없어서 `ra`, `sp`, `gp`, `tp`를 그냥 임시 변수처럼 씁니다. 하지만 C 코드와 섞이거나 OS를 만들 때는 이 규약이 중요해집니다(9장).

2.4 명령어 형식(R/I/S/B/U/J)과 immediate

모든 RV32I 명령어는 32-bit이고, 하위 7비트가 opcode입니다. opcode에 따라 나머지 비트의 해석(형식)이 정해집니다.

비트:	31	25	24	20	19	15	14	12	11	7	6	0
R형식:	funct7			rs2		rs1		funct3		rd		opcode add, sub, sll, ...
I형식:	imm[11:0]				rs1		funct3		rd		opcode addi, lw, jalr, csrr*, ecall	
S형식:	imm[11:5]			rs2		rs1		funct3		imm[4:0]		opcode sw, sh, sb
B형식:	imm[12:10:5]			rs2		rs1		funct3		imm[4:1:11]		opcode beq, bne, blt, ...
U형식:	imm[31:12]								rd		opcode lui, auipc	
J형식:	imm[20:10:1:11:19:12]								rd		opcode jal	

immediate(즉치값)는 명령어 안에 직접 박혀 있는 상수입니다. 12비트나 20비트뿐이므로 CPU가 이를 32비트로 **부호 확장(sign extension)** 해야 합니다. 부호 확장이란 최상위 비트(부호 비트)를 위쪽에 복사하는 것입니다. 예를 들어 12-bit `0xFFC` ($=-4$)는 32-bit `0xFFFFFFC` ($=-4$)가 됩니다. 모든 형식에서 부호 비트는 명령어의 bit 31에 있습니다. 그래서 `rv32_core.v`의 immediate 함수들이 전부 `{N{x[31]}}, ...`로 시작합니다(4.4.2절).

B형식과 J형식은 비트 순서가 뒤섞여 있는데, 이는 **하드웨어 배선을 단순하게** 하려는 의도입니다. 다른 형식과 최대한 같은 위치에 같은 비트를 두고, 어차피 항상 0인 bit 0을 명령어에 넣지 않아 도달 범위를 2배로 넓혔습니다.

2.5 Single-cycle, multi-cycle, pipeline

CPU 미시구조(microarchitecture)를 나누는 첫 번째 기준은 **한 명령어를 몇 개의 clock cycle에 걸쳐 처리하느냐**입니다.

- **Single-cycle(단일 사이클)**: 다섯 단계 전부를 **하나의 clock cycle** 안에 조합 논리로 끝냅니다. 매 rising edge마다 정확히 명령어 하나가 완료(retire)됩니다. 구조가 가장 단순하지만, clock 주기는 가장 느린 명령어(load: 명령어 메모리 → 디코드 → 레지스터 읽기 → 주소 덧셈 → 데이터 메모리 → 레지스터 쓰기)의 전체 경로만큼 길어야 합니다. **이 프로젝트가 이 방식입니다.** 그래서 FPGA에서 100 MHz 대신 12.5 MHz로 돌립니다.
- **Multi-cycle(다중 사이클)**: 단계마다 한 cycle씩 써서 clock을 빠르게 하되, 한 명령어에 3~5 cycle이 걸립니다.
- **Pipeline(파이프라인)**: 다섯 단계를 컨베이어 벨트처럼 겹쳐서 매 cycle 새 명령어를 시작합니다. 빠르지만 hazard(데이터 의존성, 분기) 처리가 필요합니다.

Single-cycle 구조에는 중요한 요구 조건이 하나 있습니다. **메모리가 요청한 그 cycle 안에 조합식으로 데이터를 돌려줘야** 한다는 것입니다. 주소를 주면 다음 clock에 데이터가 나오는 동기식 메모리(FPGA의 Block RAM)는 쓸 수 없습니다. 이것이 README와 rv32_soc.v 주석이 "distributed RAM으로 합성된다"고 말하는 이유입니다(8장).

2.6 메모리, little-endian, 정렬, MMIO

바이트 주소: 메모리는 바이트 단위로 주소가 붙습니다. 32-bit word 하나는 주소 4개(예: 0x400, 0x401, 0x402, 0x403)를 차지합니다.

Little-endian: RISC-V는 word의 **최하위 바이트를 가장 낮은 주소에** 둡니다. 32-bit 값 0x4B4F2043 이 주소 0x60에 있다면:

주소:	0x60	0x61	0x62	0x63
바이트:	0x43	0x20	0x4F	0x4B
문자:	'C'	' '	'O'	'K'

이것이 firmware/c_demo.hex 의 25번째 줄 4b4f2043 이 문자열 "C OK" 인 이유이고, scripts/bin2hex.py 가 `int.from_bytes(..., "little")` 로 word를 만드는 이유입니다.

정렬(alignment): 4바이트 word는 4의 배수 주소, 2바이트 halfword는 2의 배수 주소에 있는 것이 "자연 정렬"입니다. 이 프로젝트의 CPU는 자연 정렬을 가정하며, 정렬되지 않은 접근에 대한 예외(trap)를 만들지 않습니다. 컴파일러가 `-march=rv32i` 로 만든 코드는 이 규칙을 지킵니다.

Byte enable(write strobe): 메모리는 32-bit word 단위로 구성되어 있지만 sb (store byte)는 1바이트만 써야 합니다. 그래서 "이 word의 어느 바이트를 갱신할지"를 4비트 신호로 함께 보냅니다. 이것이 `dmem_wstrb[3:0]` 입니다. `4'b0001` 이면 바이트 0(최하위, 주소+0)만 씁니다.

MMIO(Memory-Mapped I/O): 주변장치(UART, timer)를 별도 명령어 없이 **특정 메모리 주소에 대한 load/store**로 제어하는 방식입니다. 주소 0x1000_0000 에 sb 를 하면 메모리가 아니라 UART가 그 바이트를 받습니다. 어느 주소가 어디로 가는지 정한 것이 **메모리 맵**입니다(3.2절).

2.7 Privileged 개념: M-mode, CSR, trap, ECALL/MRET

OS를 이해하려면 "일반 명령어 실행" 외에 **예외적인 흐름 전환**을 알아야 합니다.

특권 모드(privilege mode): RISC-V에는 M(Machine), S(Supervisor), U(User) 모드가 있습니다. M-mode는 가장 높은 권한으로 모든 하드웨어에 접근할 수 있습니다. 이 프로젝트는 **M-mode만** 구현합니다. 따라서 "OS와 사용자 프로그램의 권한 분리"는 없고, mini OS도 모든 task도 M-mode에서 돕니다.

CSR(Control and Status Register): 범용 레지스터 x0~x31과 별개인 **제어용 레지스터**입니다. 12비트 주소(최대 4096개)로 구분하고 `csrrw/csrrs/csrrc (+ i 변형)` 명령어로만 읽고 씁니다. 이 프로젝트가 구현한 CSR(`rv32_csr.v`):

주소	이름	역할
0x300	<code>mstatus</code>	전역 interrupt enable MIE (bit 3), trap 전의 MIE를 보관하는 MPIE (bit 7)
0x304	<code>mie</code>	interrupt 종류별 enable. timer용 MTIE (bit 7)만 구현
0x305	<code>mtvec</code>	trap이 발생하면 점프할 handler 주소
0x340	<code>mscratch</code>	소프트웨어가 자유롭게 쓰는 임시 저장소. trap 진입 시 레지스터 하나를 잠시 대피시키는 데 씀
0x341	<code>mepc</code>	trap이 발생한 명령어의 PC. <code>mret</code> 가 여기로 돌아감
0x342	<code>mcause</code>	trap의 원인 코드
0x344	<code>mip</code>	interrupt pending 상태. timer용 MTIP (bit 7)만, 읽기 전용
0xF11 - 0xF14	<code>mvendorid</code> 등	ID CSR, 모두 0

Trap: 정상적인 순차 실행을 **강제로 중단**하고 **handler로 점프**하는 사건의 총칭입니다. 두 종류가 있습니다.

- **Exception(예외)**: 지금 실행 중인 명령어 **자체가 원인**. 동기적(synchronous). 예: 알 수 없는 명령어(illegal instruction, cause 2), `ecall` (cause 11).
- **Interrupt(인터럽트)**: 외부 장치가 원인. 비동기적(asynchronous). 예: timer interrupt(cause `0x8000_0007` — 최상위 비트 1은 "interrupt"라는 표시, 7은 "machine timer").

Trap이 발생하면 하드웨어가 자동으로 하는 일(`rv32_csr.v` 98~102행):

1. `mepc` ← 현재 PC (어디로 돌아올지 기억)
2. `mcause` ← 원인 코드
3. `mstatus.MPIE` ← `mstatus.MIE`, `mstatus.MIE` ← 0 (handler 실행 중 또 interrupt가 걸리지 않게 끄)
4. PC ← `mtvec` (handler로 점프)

ECALL: "environment call". 소프트웨어가 **의도적으로** trap을 일으키는 명령입니다. OS에서는 **system call**의 진입점으로 씁니다. 사용자 코드가 `ecall` 을 실행하면 OS의 trap handler가 실행되고, 어떤 서비스를 원하는지는 레지스터(관례적으로 `a7`)로 전달합니다.

MRET: "machine return". trap handler가 끝나면 실행합니다. 하드웨어가 하는 일(`rv32_csr.v` 105~107행, `rv32_core.v` 250행):

1. PC ← `mepc`
2. `mstatus.MIE` ← `mstatus.MPIE` (interrupt enable 복구), `mstatus.MPIE` ← 1

중요한 차이: interrupt로 진입했다면 mepc는 "아직 실행되지 못한 명령어"이므로 그대로 돌아가면 됩니다. 하지만 ecall로 진입했다면 mepc는 ecall 자신의 주소이므로, 그대로 돌아가면 ecall이 또 실행되어 무한 반복됩니다. 따라서 handler가 mepc += 4를 해 줘야 합니다. 9장에서 다시 다룹니다.

2.8 이 프로젝트를 읽는 데 필요한 Verilog 최소 지식

RTL 파일을 읽기 위해 필요한 문법만 정리합니다.

```
`timescale 1ns/1ps          // 시뮬레이션 시간 단위 1ns, 정밀도 1ps. "#5"는 5ns.
`default_nettype none       // 선언하지 않은 이름을 쓰면 오류. 오타 방지용. 파일 끝에서 wire로 복구.

module 이름 #(parameter P = 값) ( input wire a, output reg b ); // parameter는 인스턴스마다 바꿀 수
있는 상수
```

- **wire**: 조합 논리 배선. assign w = a & b; 처럼 항상 식으로 정의됩니다.
- **reg**: always 블록 안에서 값을 대입하는 신호. 이름과 달리 반드시 flip-flop이 되는 것은 아닙니다. always @* 안에서 대입하면 조합 논리, always @(posedge clk) 안에서 대입하면 flip-flop입니다.
- **always @***: 조합 논리 블록. 입력이 바뀌면 즉시 다시 계산. 여기서는 **blocking 대입 =** 을 씁니다. 모든 경로에서 모든 출력에 값을 주지 않으면 합성기가 "이전 값을 기억"하는 latch를 만들어 버립니다. 그래서 rv32_core.v는 블록 첫머리에서 모든 출력에 기본값을 줍니다(144~148행).
- **always @(posedge clk)**: clock의 rising edge에서만 동작하는 순차 논리(flip-flop). **non-blocking 대입 <=** 을 씁니다. 블록 안의 모든 <= 는 "edge 순간의 오른쪽 값"으로 동시에 갱신됩니다. 같은 신호에 두 번 <= 하면 뒤의 것이 이깁니다(simple_timer.v 48~51행에서 이용).
- **case / if**: 하드웨어에서는 mux(선택기)가 됩니다.
- **{a, b}**: 비트 이어붙이기(concatenation). {{20{x[31]}}, x[31:20]}는 "x[31]을 20번 반복한 뒤 x[31:20]을 붙임" = 부호 확장.
- **|dwstrb**: 단항(reduction) OR. 4비트 중 하나라도 1이면 1.
- **\$signed(a) < \$signed(b)**: 부호 있는 비교. 기본은 부호 없는 비교입니다.
- **>>>**: 산술 우측 시프트(부호 비트 채움). >> 는 논리 시프트(0 채움).
- **reg [31:0] mem [0:2047];**: 32-bit 원소 2048개짜리 메모리 배열.
- **\$readmemh("파일", mem)**: 16진수 텍스트 파일을 배열에 적재. 시뮬레이터와 Vivado 합성 모두 지원합니다(FPGA에서는 메모리 초기값이 됨).
- **initial**: 시뮬레이션 시작(또는 FPGA 설정) 시 한 번 실행.
- **dut.cpu.rf.regs[15]**: 계층적 참조. 테스트벤치가 내부 신호를 들여다볼 때 씁니다.
- **!==**: 4-state 비교(X, Z까지 비교). 테스트벤치 검사에 씁니다.

3. 하드웨어 전체 구조

3.1 모듈 계층

실제 인스턴스 계층은 다음과 같습니다(FPGA 기준).

```
nexys_a7_top      (rtl/nexys_a7_top.v)    100 MHz / 12.5 MHz 두 clock domain
├─ cpu_clk_buf : BUFG      Xilinx global clock buffer (12.5 MHz)
├─ soc : rv32_soc          (rtl/rv32_soc.v)    12.5 MHz domain
│   ├── mem[0:2047]        8 KiB 통합 instruction/data memory
│   ├── timer : simple_timer (rtl/simple_timer.v) mtime/mtimecmp
│   └── cpu : rv32_core     (rtl/rv32_core.v)    단일 사이클 RV32I
│       ├── rf : rv32_regfile (rtl/rv32_regfile.v) x1..x31
│       ├── alu : rv32_alu    (rtl/rv32_alu.v)
│       └── csr : rv32_csr    (rtl/rv32_csr.v)
├─ mstatus/mie/mtvec/mepc/mcause/mscratch
└─ serial : uart_tx        (rtl/uart_tx.v)      100 MHz domain, 115200 baud
```

시뮬레이션(`tb_rv32_core.v`, `tb_c_demo.v`)은 `nexys_a7_top` 대신 `rv32_soc`를 직접 인스턴스하고, `uart_tx_ready`를 상수 1로 묶어 UART 송신기 없이 바이트 스트림만 관찰합니다.

3.2 메모리 맵

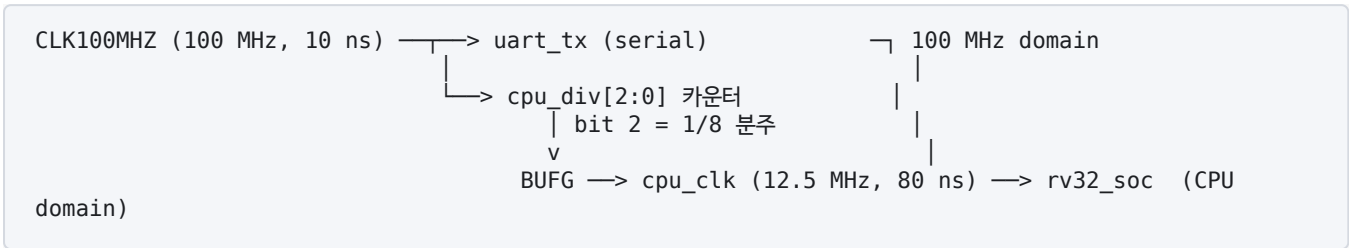
CPU가 보는 32-bit 주소 공간에서 실제로 무엇인가가 연결된 곳은 다음뿐입니다. 디코딩은 전부 `rv32_soc.v`에서 합니다.

주소 범위	장치	읽기	쓰기	디코딩 조건(<code>rv32_soc.v</code>)
0x0000_0000 - 0x0000_1FFF (FPGA, 8 KiB) 0x0000_0000 - 0x0000_FFFF (SoC 기본값, 64 KiB)	통합 RAM	명령어 fetch + 데이터	byte enable 지원	<code>addr < MEM_WORDS*4</code>
0x1000_0000	UART TX 데이터	0	하위 8비트 송신 (ready일 때만 수락)	<code>daddr == 0x1000_0000</code>
0x1000_0004	UART TX ready	bit 0 = 송신 가능	무시	<code>daddr == 0x1000_0004</code>
0x1000_1000	<code>mtime[31:0]</code>	현재 시각	갱신 가능	<code>daddr[31:12] == 0x10001</code> 이고 <code>addr[2]==0</code>
0x1000_1004	<code>mtimecmp[31:0]</code>	비교값	갱신 → IRQ 해제/ 예약	<code>daddr[31:12] == 0x10001</code> 이고 <code>addr[2]==1</code>
그 외	없음	데이터 read는 0, 명령어 fetch는 NOP(0x13)	무시	—

메모리 크기는 `rv32_soc`의 `MEM_WORDS` parameter로 정해집니다. 기본값은 16384 words(64 KiB)이지만 `nexys_a7_top.v` 68행에서 2048 words(8 KiB)로 override하고, `tb_c_demo.v`도 2048을 씁니다.

firmware/linker.ld의 LENGTH = 8K와 bin2hex.py --max-bytes 8192 는 이 8 KiB에 맞춘 것입니다. 세 곳 (top, linker, Makefile)이 같은 크기를 가리켜야 합니다.

3.3 두 개의 clock domain (FPGA)



CPU가 12.5 MHz인 이유는 2.5절에서 설명한 single-cycle 경로 길이 때문입니다. UART는 115200 baud 분주에 100 MHz가 더 정확해서 원래 clock을 그대로 씁니다. 두 clock 사이를 넘어가는 신호(uart_valid, uart_data, uart_ready)의 처리는 4.8절에서 설명합니다.

3.4 코어와 SoC 사이의 인터페이스 계약

rv32_core.v 11~17행 주석에 명시된 계약입니다. 이 계약을 이해하면 나중에 메모리를 바꿀 때 무엇을 지켜야 하는지 알 수 있습니다.

신호	방향 (core 기준)	의미
imem_addr[31:0]	out	항상 현재 PC
imem_rdata[31:0]	in	같은 cycle에 조합식으로 돌아와야 하는 명령어
dmem_addr[31:0]	out	바이트 주소. load/store가 아니면 0
dmem_rdata[31:0]	in	주소가 속한 정렬된 32-bit word 전체. 바이트 고르기는 core가 함
dmem_wdata[31:0]	out	쓸 데이터. sb / sh 는 바이트/하프워드가 4번/2번 복제되어 있음
dmem_wstrb[3:0]	out	바이트 lane별 write enable. 0000이면 쓰기 없음
timer_irq	in	level(레벨) 신호. 1인 동안 계속 요청
debug_pc, trap_taken	out	관측용

ready/valid 같은 handshake가 없으므로 대기 상태(wait state)가 필요한 느린 메모리는 직접 연결할 수 없습니다.

4. RTL 상세 설명

각 파일을 "역할 → 인터페이스 → 코드 블록별 설명 → 주의점" 순서로 설명합니다. 코드는 저장소의 실제 내용을 인용하며, 행 번호는 해당 파일 기준입니다.

4.1 rtl/rv32_alu.v — 조합식 산술/논리 연산기

역할: 두 32-bit 입력 `a`, `b`와 4-bit 연산 코드 `op`를 받아 같은 cycle에 결과 `y`를 냅니다. clock도 reset도 없는 순수 조합 논리입니다.

```
module rv32_alu (
    input wire [3:0] op,
    input wire [31:0] a,
    input wire [31:0] b,
    output reg [31:0] y
);
    localparam ALU_ADD = 4'd0, ALU_SUB = 4'd1,
               ALU_SLL = 4'd2, ALU_SLT = 4'd3,
               ALU_SLTU = 4'd4, ALU_XOR = 4'd5,
               ALU_SRL = 4'd6, ALU_SRA = 4'd7,
               ALU_OR = 4'd8, ALU_AND = 4'd9;
```

`op`의 인코딩은 ISA의 funct3/funct7이 아니라 core 내부에서만 쓰는 축약 코드입니다. `rv32_core.v` 52~54행에 똑같은 `localparam`이 있고, 두 곳이 반드시 일치해야 합니다. core가 ISA 인코딩을 이 4-bit 코드로 번역해 주는 구조입니다.

```
always @* begin
    case (op)
        ALU_ADD: y = a + b;
        ALU_SUB: y = a - b;
        ALU_SLL: y = a << b[4:0];
        ALU_SLT: y = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
        ALU_SLTU: y = (a < b) ? 32'd1 : 32'd0;
        ALU_XOR: y = a ^ b;
        ALU_SRL: y = a >> b[4:0];
        ALU_SRA: y = $signed(a) >>> b[4:0];
        ALU_OR: y = a | b;
        ALU_AND: y = a & b;
        default: y = 32'd0;
    endcase
end
```

각 줄에서 주의 깊게 볼 점:

- **b[4:0]만 사용하는 시프트:** RV32에서 시프트 양(shamt)은 0~31이므로 하위 5비트만 의미가 있습니다. `slli`의 immediate나 `sll`의 `rs2` 값이 32 이상이어도 하위 5비트만 적용하는 것이 ISA 규칙입니다.
- **SLT와 SLTU의 분리:** Verilog의 `<`는 기본이 unsigned 비교입니다. 예를 들어 `a = 0xFFFFFFFF` (signed로 -1), `b = 1`이면 signed로는 `a < b`가 참이지만 unsigned로는 거짓입니다. `$signed()` 캐스트로 명시합니다.
- **SRA의 \$signed(a) >>> b:** >>> 만 쓰면 왼쪽 피연산자가 unsigned 타입일 때 0으로 채워집니다. `$signed(a)`로 감싸야 비로소 부호 비트가 복사됩니다.
- **default: y = 0:** 정상 디코드에서는 10개 코드만 오지만, 모든 경우에 `y`를 대입해야 latch가 생기지 않습니다.
- **오버플로 검출 없음:** RISC-V 정수 덧셈은 오버플로 예외를 정의하지 않으므로 carry-out을 버립니다.

4.2 rtl/rv32_regfile.v — 2R1W 레지스터 파일

역할: x1 ~ x31 을 저장하고, 두 소스 레지스터를 조합식으로 읽으며, rising edge에서 목적지 레지스터 하나를 씁니다.

```
reg [31:0] regs [1:31];
```

배열 범위가 [1:31] 입니다. **x0** 는 물리적 저장소 자체를 만들지 않습니다. 대신 읽기 쪽에서 처리합니다:

```
assign rs1_data = (rs1_addr == 5'd0) ? 32'd0 : regs[rs1_addr];
assign rs2_data = (rs2_addr == 5'd0) ? 32'd0 : regs[rs2_addr];
```

읽기는 assign (조합 논리)입니다. 주소를 바꾸면 즉시 값이 나옵니다. single-cycle에서는 "이번 cycle에 fetch한 명령어의 rs1을 이번 cycle에 읽어야" 하므로 필수입니다.

```
always @(posedge clk) begin
    if (rst) begin
        for (i = 1; i < 32; i = i + 1)
            regs[i] <= 32'd0;
    end else if (rd_we && (rd_addr != 5'd0)) begin
        regs[rd_addr] <= rd_data;
    end
end
```

- 쓰기는 동기식이며 rd_addr == 0 이면 버립니다. core도 rd_we 를 만들 때 x0를 따로 검사하지 않고 이 모듈에 맡깁니다.
- reset 시 모두 0으로 초기화합니다. RISC-V ISA는 reset 후 레지스터 값을 정의하지 않지만, 시뮬레이션의 X 전파를 막고 FPGA에서 결정적으로 동작하게 하려는 교육용 선택입니다. 대가로 이 배열은 FPGA에서 distributed RAM이 아니라 flip-flop $31 \times 32 = 992$ 개로 구현됩니다(reset 가능한 RAM은 없기 때문). 8장의 register 사용량 1,372개 중 대부분이 여기입니다.
- 같은 cycle 읽기/쓰기 bypass 없음: 명령어 N이 rising edge에서 x5 를 쓰고, 명령어 N+1은 그 edge 이후에 x5 를 읽습니다. single-cycle에서는 edge를 사이에 두므로 forwarding이 필요 없습니다. pipeline으로 바꾸면 이 가정이 깨집니다.

4.3 rtl/rv32_csr.v — M-mode CSR와 trap 상태

역할: 2.7절의 CSR들을 저장하고, (1) CSR 명령의 읽기/쓰기, (2) trap 진입 시 자동 갱신, (3) mret 시 복구, (4) timer interrupt pending 판단을 담당합니다.

4.3.1 인터페이스

포트	의미
read_addr[11:0], read_data, read_valid	조합식 읽기. read_valid=0 이면 존재하지 않는 CSR → core가 illegal instruction 처리
write_en, write_addr, write_data	CSR 명령에 의한 쓰기(rising edge)
trap_enter, trap_pc, trap_cause	trap 진입 요청. core가 take_trap 을 넣어줌
mret	MRET 실행 알림
timer_irq	timer의 level 신호

포트	의미
irq_pending	"지금 interrupt를 받아야 한다"는 최종 판단
mtvec, mepc	core가 PC를 바꿀 때 쓰는 값

4.3.2 핵심 조합 논리

```
assign mtvec = {mtvec_r[31:2], 2'b00}; // direct mode only
assign mepc = {mepc_r[31:2], 2'b00};
assign irq_pending = mstatus_r[3] && mie_r[7] && timer_irq;
```

- mtvec 의 하위 2비트는 표준에서 "mode"(0=direct, 1=vectored)입니다. 이 구현은 direct mode만 지원하므로 하위 2비트를 항상 0으로 만듭니다. handler는 mtvec 한 곳으로만 갑니다.
- irq_pending 식이 가장 중요합니다. timer가 요청(timer_irq=1)하더라도 전역 enable mstatus.MIE (bit 3) 와 timer enable mie.MTIE (bit 7)가 모두 1이어야 CPU가 반응합니다. reset 직후에는 둘 다 0이므로 소프트웨어가 켜기 전까지 interrupt는 절대 발생하지 않습니다.

4.3.3 읽기 mux

```
always @* begin
    read_valid = 1'b1;
    case (read_addr)
        12'h300: read_data = mstatus_r;
        12'h304: read_data = mie_r;
        12'h305: read_data = mtvec_r;
        12'h340: read_data = mscratch_r;
        12'h341: read_data = mepc_r;
        12'h342: read_data = mcause_r;
        12'h344: read_data = timer_irq ? 32'h0000_0080 : 32'd0;
        12'hF11: read_data = 32'd0; // mvendorid
        12'hF12: read_data = 32'd0; // marchid
        12'hF13: read_data = 32'd0; // mimpid
        12'hF14: read_data = 32'd0; // mhartid
        default: begin read_data = 32'd0; read_valid = 1'b0; end
    endcase
end
```

- mip (0x344)는 저장소가 없습니다. timer_irq 선을 그대로 bit 7에 비춰 줍니다. 소프트웨어가 mip 를 써도 아래 쓰기 로직의 default: ; 에 걸려 무시됩니다(에러도 아님).
- mhartid 가 0인 것은 "0번 hart(하드웨어 스레드)"라는 의미로, 단일 코어에서는 표준적인 값입니다.
- read_addr 은 core에서 insn[31:20] 으로 모든 명령어에 대해 연결되어 있습니다. CSR 명령이 아닐 때는 그저 무시되는 값입니다.

4.3.4 순차 논리: 쓰기, trap 진입, MRET

```
always @(posedge clk) begin
    if (rst) begin
        mstatus_r <= 0; mie_r <= 0; mtvec_r <= 0; mscratch_r <= 0; mepc_r <= 0; mcause_r <= 0;
    end else begin
        if (write_en) begin
            case (write_addr)
                12'h300: mstatus_r <= (mstatus_r & ~32'h0000_0088) | (write_data & 32'h0000_0088);
                12'h304: mie_r <= write_data & 32'h0000_0080;
```

```

12'h305: mtvec_r    <= {write_data[31:2], 2'b00};
12'h340: mscratch_r <= write_data;
12'h341: mepc_r     <= {write_data[31:2], 2'b00};
12'h342: mcause_r   <= write_data;
default: ;
endcase
end
if (trap_enter) begin
    mepc_r        <= trap_pc;
    mcause_r       <= trap_cause;
    mstatus_r[7] <= mstatus_r[3]; // MIE <- MIE
    mstatus_r[3] <= 1'b0;         // MIE <- 0
end else if (mret) begin
    mstatus_r[3] <= mstatus_r[7]; // MIE <- MIE
    mstatus_r[7] <= 1'b1;
end
end
end

```

- **마스크 0x88**: mstatus 에서 bit 3(MIE)과 bit 7(MPIE)만 쓸 수 있게 합니다. 나머지 비트(MPP 등)는 0으로 고정됩니다. mie 는 bit 7(MTIE)만 남깁니다. 이렇게 "구현하지 않은 비트는 0으로 읽힘(WARL)"이 표준 관행입니다.
- **우선순위**: 같은 edge에 write_en 과 trap_enter 가 동시에 참이면, 코드상 뒤에 있는 trap_enter 블록의 non-blocking 대입이 이깁니다. 다만 core는 trap이 선택된 cycle에 write_en 을 이미 0으로 막으므로(csr_we && !take_trap) 실제로는 겹치지 않습니다. trap_enter 가 mret 보다 우선하는 것도 같은 구조입니다.
- **MPIE ← 1 in MRET**: 표준이 정한 동작입니다. 덕분에 handler가 mstatus 를 명시적으로 조작하지 않아도 새 task를 시작할 때 interrupt가 켜진 상태로 진입할 수 있습니다(9장에서 사용).

4.4 rtl/rv32_core.v — 단일 사이클 RV32I 코어

이 파일이 프로세서의 중심입니다. 약 290행이지만 구조는 단순합니다: (1) 명령어 필드 분해, (2) immediate 함수, (3) 거대한 always @* 디코더, (4) PC 업데이트 한 줄.

4.4.1 명령어 필드와 내부 신호 (59~93행)

```

reg [31:0] pc;
wire [31:0] insn = imem_rdata;
wire [6:0] opcode = insn[6:0];
wire [2:0] funct3 = insn[14:12];
wire [6:0] funct7 = insn[31:25];
wire [4:0] rs1 = insn[19:15], rs2 = insn[24:20], rd = insn[11:7];

```

2.4절의 형식 그림과 정확히 대응합니다. pc 가 이 모듈의 **유일한 flip-flop**입니다(레지스터 파일과 CSR은 하위 모듈에 있음). 나머지 reg 들은 모두 always @* 안에서 대입되는 조합 신호입니다.

```

wire take_trap = irq_pending || exception;
wire [31:0] trap_cause = irq_pending ? 32'h8000_0007 : exception_cause;

```

- take_trap 은 "이번 cycle에 trap으로 들어간다"입니다. interrupt가 exception보다 우선합니다. 이 cycle의 명령어는 실행되지 않은 것으로 취급되어 mepc 에 그 PC가 저장되므로, MRET 후 다시 실행됩니다.

4.4.2 부작용(side effect) 차단 (98~124행)

```
assign dmem_wstrb = take_trap ? 4'b0000 : dwstrb_r;
rv32_regfile rf ( ... .rd_we(rd_we && !take_trap), ... );
rv32_csr csr ( ... .write_en(csr_we && !take_trap), ...
               .trap_enter(take_trap), .trap_pc(pc), .trap_cause(trap_cause),
               .mret(do_mret && !take_trap), ... );
```

trap이 선택된 cycle에 레지스터 쓰기, 메모리 쓰기, CSR 쓰기, MRET 효과가 모두 억제됩니다. 이 네 줄이 "trap이 걸린 명령어는 아키텍처 상태를 바꾸지 않는다"는 정확성 요구를 구현합니다. 예를 들어 sw 실행 도중 timer interrupt가 걸리면 store는 일어나지 않고, MRET 후 sw가 다시 실행되어 그때 store가 일어납니다.

4.4.3 Immediate 함수 (131~135행)

```
function [31:0] imm_i; input [31:0] x; imm_i={{20{x[31]}},x[31:20]}; endfunction
function [31:0] imm_s; input [31:0] x; imm_s={{20{x[31]}},x[31:25],x[11:7]}; endfunction
function [31:0] imm_b; input [31:0] x; imm_b=
{{19{x[31]}},x[31],x[7],x[30:25],x[11:8],1'b0}; endfunction
function [31:0] imm_u; input [31:0] x; imm_u={x[31:12],12'd0}; endfunction
function [31:0] imm_j; input [31:0] x; imm_j=
{{11{x[31]}},x[31],x[19:12],x[20],x[30:21],1'b0}; endfunction
```

각 함수는 2.4절 형식 그림의 비트를 제자리에 놓고 부호 확장하는 배선입니다. 비트 수를 세어 보면:

함수	구성	비트 수
imm_i	부호 20 + [31:20] 12	32
imm_s	부호 20 + [31:25] 7 + [11:7] 5	32
imm_b	부호 19 + [31] 1 + [7] 1 + [30:25] 6 + [11:8] 4 + 0 1	32
imm_u	[31:12] 20 + 0 12	32
imm_j	부호 11 + [31] 1 + [19:12] 8 + [20] 1 + [30:21] 10 + 0 1	32

B/J의 마지막 1'b0이 "항상 짝수 offset"을 만듭니다. Verilog function은 합성 시 그냥 배선으로 펼쳐지므로 비용이 없습니다.

4.4.4 디코더 기본값 (143~148행)

```
always @* begin
  rd_we = 1'b0; rd_data = 32'd0; next_pc = pc + 32'd4;
  alu_op = ALU_ADD; alu_a = rs1_data; alu_b = rs2_data;
  daddr_r = 32'd0; dwdata_r = 32'd0; dwstrb_r = 4'd0; load_shifted = 32'd0;
  illegal = 1'b0; exception = 1'b0; exception_cause = 32'd0;
  do_mret = 1'b0; csr_we = 1'b0; csr_waddr = insn[31:20]; csr_wdata = 32'd0;
```

"가장 흔한 경우"를 기본값으로 둡니다: 다음 PC는 PC+4, 아무것도 쓰지 않음, ALU는 rs1 + rs2. 각 opcode 분기는 바꿔야 하는 것만 덮어씁니다. 2.8절에서 말한 latch 방지 관행입니다.

4.4.5 opcode별 동작

LUI / AUIPC (152~153행)

```
7'b0110111: begin rd_we=1'b1; rd_data=imm_u(insn); end // LUI
7'b0010111: begin rd_we=1'b1; rd_data=pc+imm_u(insn); end // AUIPC
```

lui rd, imm 은 상위 20비트를 채웁니다. lui x1, 0x10000 은 x1 = 0x1000_0000 (UART base)입니다. 32-bit 상수는 lui + addi 두 명령으로 만듭니다(c_demo.s 의 %hi / %lo 가 그것). auipc 는 PC 기준 상대 주소를 만들어 위치 독립 코드에 씁니다.

JAL (156행)

```
7'b1101111: begin rd_we=1'b1; rd_data=pc+4; next_pc=pc+imm_j(insn); end
```

반환 주소 pc+4 를 rd 에 저장하고 pc + offset 으로 점프합니다. rd=x0 이면 저장이 버려져 순수 점프(j)가 됩니다. nexys_hello.s 의 j 1b, c_demo 의 jal x0,5c (자기 자신으로 무한 점프)가 이 경우입니다.

JALR (159~162행)

```
7'b1100111: begin
    if (funct3 != 3'b000) illegal=1'b1;
    else begin rd_we=1'b1; rd_data=pc+4; next_pc=
(rs1_data+imm_i(insn))&32'hffff_fffe; end
end
```

레지스터 간접 점프. 함수에서 돌아올 때 (ret = jalr x0, 0(ra)) 씁니다. 목적지의 bit 0을 강제로 0으로 만드는 것 (& 0xFFFF_FFFE)은 ISA 규정입니다.

Branch (164~174행)

```
7'b1100011: begin
    case (funct3)
        3'b000: if (rs1_data == rs2_data) next_pc=pc+imm_b(insn);
// BEQ
        3'b001: if (rs1_data != rs2_data) next_pc=pc+imm_b(insn);
// BNE
        3'b100: if ($signed(rs1_data) < $signed(rs2_data)) next_pc=pc+imm_b(insn);
// BLT
        3'b101: if ($signed(rs1_data) >= $signed(rs2_data))
next_pc=pc+imm_b(insn); // BGE
        3'b110: if (rs1_data < rs2_data) next_pc=pc+imm_b(insn);
// BLTU
        3'b111: if (rs1_data >= rs2_data) next_pc=pc+imm_b(insn);
// BGEU
        default: illegal=1'b1;
    endcase
end
```

조건이 거짓이면 아무것도 덮어쓰지 않아 기본값 pc+4 가 유지됩니다. 비교기는 ALU를 거치지 않고 core에서 직접 만듭니다. funct3=010, 011 은 정의되지 않은 인코딩이라 illegal입니다. c_demo 의 반복문 종료 검사 bgeu x12,x14,18 (sum_limit >= i 이면 계속)이 3'b111 입니다.

LOAD (178~189행)

```
7'b0000011: begin
    daddr_r=rs1_data+imm_i(insn); rd_we=1'b1;
    load_shifted=dmem_rdata >> (8*daddr_r[1:0]);
    case (funct3)
```

```

3'b000: rd_data={{24{load_shifted[7]}},load_shifted[7:0]}; // LB
3'b001: rd_data={{16{load_shifted[15]}},load_shifted[15:0]}; // LH
3'b010: rd_data=dmem_rdata; // LW
3'b100: rd_data={24'd0,load_shifted[7:0]}; // LBU
3'b101: rd_data={16'd0,load_shifted[15:0]}; // LHU
default: begin illegal=1'b1; rd_we=1'b0; end
endcase
end

```

동작 순서(모두 한 cycle 안의 조합 논리):

1. 유효 주소 = `rs1 + imm_i`. 이 값이 `dmem_addr` 로 나가고 SoC가 그 주소가 속한 word 전체를 `dmem_rdata` 로 돌려줍니다.
2. `daddr_r[1:0]` (word 안에서의 바이트 위치, 0~3)에 8을 곱한 만큼 오른쪽으로 시프트하면 원하는 바이트가 bit [7:0]에 옵니다. 예: 주소 0x61 (' ')이면 `0x4B4F2043 >> 8 = 0x004B4F20`, 하위 바이트 0x20.
3. `funct3`에 따라 부호 확장(LB/LH) 또는 0 확장(LBU/LHU)합니다. `nexys_hello.S`가 `lbu`를 쓰는 이유: 문자 코드 0x80 이상이 음수로 확장되지 않게 하려는 것입니다.

STORE (192~200행)

```

7'b0100011: begin
    daddr_r=rs1_data+imm_s(insn);
    case (funct3)
        3'b000: begin dwstrb_r=4'b0001 << daddr_r[1:0]; dwd_data_r=
{4{rs2_data[7:0]}}; end // SB
        3'b001: begin dwstrb_r=4'b0011 << daddr_r[1:0]; dwd_data_r=
{2{rs2_data[15:0]}}; end // SH
        3'b010: begin dwstrb_r=4'b1111; dwd_data_r=rs2_data; end
// SW
        default: illegal=1'b1;
    endcase
end

```

바이트 복제 기법이 핵심입니다. `sb`는 8비트 값을 네 lane 모두에 복사하고(`{4{...}}`), byte enable만 목표 lane 하나를 컷니다. 그러면 메모리가 "켜진 lane의 데이터"만 받아들이므로 어느 위치든 올바른 바이트가 기록됩니다. 별도의 시프터가 필요 없습니다. `c_demo`가 UART에 'C' (0x43)를 `sb`로 쓸 때 `dmem_wdata`가 0x43434343으로 보이는 이유입니다(VCD에서 확인 가능).

OP-IMM (203~217행)

```

7'b0010011: begin
    rd_we=1'b1; alu_b=imm_i(insn);
    case (funct3)
        3'b000: alu_op=ALU_ADD; 3'b010: alu_op=ALU_SLT;
        3'b011: alu_op=ALU_SLTU; 3'b100: alu_op=ALU_XOR;
        3'b110: alu_op=ALU_OR; 3'b111: alu_op=ALU_AND;
        3'b001: begin alu_op=ALU_SLL; if (funct7!=7'b0000000) illegal=1'b1; end
        3'b101: begin
            if (funct7==7'b0000000) alu_op=ALU_SRL;
            else if (funct7==7'b0100000) alu_op=ALU_SRA;
            else illegal=1'b1;
        end
    endcase
    rd_data=alu_y; if (illegal) rd_we=1'b0;
end

```

두 번째 ALU 입력을 rs2 대신 immediate로 바꾼 것뿐입니다. srl이와 srai는 funct3가 같고(101) immediate의 상위 7비트(=funct7 자리)로 구분됩니다. addi x0,x0,0 (= 0x00000013)이 표준 NOP이며, SoC가 범위 밖 fetch에 이 값을 돌려줍니다.

OP (219~235행)

```
7'b0110011: begin
  rd_we=1'b1;
  case ({funct7, funct3})
    {7'b0000000, 3'b000}: alu_op=ALU_ADD;
    {7'b0100000, 3'b000}: alu_op=ALU_SUB;
    ...
    default: illegal=1'b1;
  endcase
  rd_data=alu_y; if (illegal) rd_we=1'b0;
end
```

레지스터-레지스터 연산. {funct7, funct3} 10비트를 한 번에 비교합니다. add와 sub는 funct7 bit 5로만 다릅니다. 곱셈(mul, funct7=0000001)은 M 확장이므로 여기 없고 illegal이 됩니다. 그래서 -march=rv32i가 필수입니다. rv32im으로 컴파일하면 컴파일러가 mul을 내보내고 CPU는 illegal instruction trap을 일으킵니다.

MISC-MEM / FENCE (238행)

```
7'b0001111: begin if (funct3!=3'b000) illegal=1'b1; end
```

fence는 메모리 순서를 보장하는 명령입니다. 이 CPU는 한 번에 한 접근만 하고 완료를 기다리므로 순서가 항상 지켜져 NOP으로 충분합니다.

SYSTEM: ECALL / MRET / CSR (245~269행)

```
7'b1110011: begin
  if (funct3 == 3'b000) begin
    if (insn == 32'h0000_0073) begin exception=1'b1; exception_cause=32'd11;
  end // ECALL
    else if (insn == 32'h3020_0073) begin do_mret=1'b1; next_pc=csr_mepc; end
  // MRET
    else illegal=1'b1;
  end else begin
    if (!csr_valid) illegal=1'b1;
    else begin
      rd_we=1'b1; rd_data=csr_rdata;
      case (funct3)
        3'b001: begin csr_we=1'b1; csr_wdata=rs1_data; end
        // CSRRW
        3'b010: begin csr_we=(rs1!=0); csr_wdata=csr_rdata|rs1_data; end
        // CSRRS
        3'b011: begin csr_we=(rs1!=0); csr_wdata=csr_rdata&~rs1_data; end
        // CSRRC
        3'b101: begin csr_we=1'b1; csr_wdata={27'd0,rs1}; end
        // CSRRWI
        3'b110: begin csr_we=(rs1!=0); csr_wdata=csr_rdata|{27'd0,rs1}; end
        // CSRRSI
        3'b111: begin csr_we=(rs1!=0); csr_wdata=csr_rdata&~{27'd0,rs1};
      end // CSRRCI
      default: begin illegal=1'b1; csr_we=1'b0; rd_we=1'b0; end
    endcase
  end
end
end
```

- `ecall` 은 전체 32비트가 `0x00000073` 으로 고정된 명령입니다. `exception=1`, `cause 11` ("environment call from M-mode")을 세우면 `take_trap` 이 1이 됩니다.
- `mret` (`0x30200073`)는 `next_pc = mepc` 로 바꾸고 CSR 블록에 `mret` 를 알립니다. `ebreak` (`0x00100073`)와 `wfi` 는 구현하지 않아 illegal입니다.
- CSR 명령은 항상 이전 CSR 값을 `rd` 에 돌려주고, 종류에 따라 새 값을 계산합니다: RW(덮어쓰기), RS(비트 set = OR), RC(비트 clear = AND NOT). I 변형은 `rs1` 필드 5비트를 상수(zimm)로 씁니다. 예: `csrsi mstatus`, `0x8` 은 "MIE 비트 켜기"입니다.
- RS/RC에서 `rs1 == x0` 이면 쓰기를 하지 않습니다. 이것이 표준의 "csrr rd, csr = csrrs rd, csr, x0는 읽기 전용" 규칙입니다. 읽기 전용 CSR(`mip`)에 대한 부작용을 피하는 데 중요합니다.
- 존재하지 않는 CSR(`csr_valid=0`)은 illegal instruction입니다.

illegal 마무리 (276행)

```
if (illegal) begin exception=1'b1; exception_cause=32'd2; csr_we=1'b0; rd_we=1'b0;
dwstrb_r=4'd0; end
```

어느 분기에서든 `illegal` 이 세워졌다면 `cause 2 exception`으로 통일하고, 그 분기가 이미 켜 두었는지 모르는 쓰기 신호를 모두 끕니다.

4.4.6 PC 업데이트 (284~288행)

```
always @(posedge clk) begin
  if (rst) pc <= RESET_PC;
  else if (take_trap) pc <= csr_mtvec;
  else pc <= next_pc;
end
```

우선순위는 `reset` > `trap` > 정상 `next_pc` 입니다. `RESET_PC` 기본값 0이 곧 "reset 벡터"이며, `linker.ld`가 `_start` 를 0번지에 놓는 이유입니다.

4.5 rtl/simple_timer.v — machine timer

역할: 매 clock 1씩 증가하는 64-bit `mtime` 과, 비교 기준 `mtimecmp` 를 가지고, `mtime >= mtimecmp` 인 동안 `irq` 를 1로 유지합니다.

```
reg [63:0] mtime;
reg [63:0] mtimecmp;
assign irq = (mtime >= mtimecmp);
```

- **level 신호:** 등호가 성립하는 한 순간이 아니라 "이상"인 동안 계속 1입니다. CPU가 그 순간을 놓쳐도(예: interrupt가 disable 상태) 나중에 반응할 수 있습니다. 대신 `handler`가 `mtimecmp` 를 미래로 옮겨서 직접 꺼야 합니다. 그렇지 않으면 MRET 직후 다시 interrupt가 걸립니다.

```
always @* begin
  case (addr[2])
    1'b0: rd_data = mtime[31:0];
    1'b1: rd_data = mtimecmp[31:0];
  endcase
end
```

```

always @(posedge clk) begin
    if (rst) begin
        mtime <= 64'd0;
        mtimecmp <= 64'h0000_0000_ffff_ffff;
    end else begin
        mtime <= mtime + 64'd1;
        if (wr_en && !addr[2]) mtime[31:0] <= wr_data;
        if (wr_en && addr[2]) mtimecmp[31:0] <= wr_data;
    end
end
end

```

- reset 시 mtimecmp = 0xFFFF_FFFF 로 두어 소프트웨어가 값을 쓰기 전에는 IRQ가 나지 않게 합니다.
- mtime <= mtime + 1 다음에 mtime[31:0] <= wr_data 가 오므로, 같은 edge에 쓰기가 있으면 쓰기가 이깁니다(non-blocking 대입의 "마지막이 이긴다" 규칙).
- 하위 32비트만 MMIO로 노출됩니다. 상위 32비트는 mtime 은 계속 증가하지만 mtimecmp 의 상위는 항상 0입니다. 따라서 mtime 이 2^{32} 을 넘는 순간(12.5 MHz에서 약 343초 $\approx$ 5분 43초) mtime >= mtimecmp 가 항상 참이 되어 IRQ가 영구히 켜집니다. 짧은 데모에는 충분하지만, 장시간 실행하는 OS에는 상위 word 레지스터를 추가해야 합니다(9.7절).

tick 계산: FPGA에서 CPU clock이 12.5 MHz이므로 1 tick = 80 ns, 10 ms = 125,000 tick, 1초 = 12,500,000 tick입니다. 시뮬레이션(tb_c_demo.v)에서는 clock 주기가 10 ns이므로 1 tick = 10 ns입니다.

4.6 rtl/rv32_soc.v — 메모리와 MMIO를 묶은 SoC

4.6.1 메모리 선언과 초기화 (34~41행)

```

reg [31:0] mem [0:MEM_WORDS-1];
integer i;
initial begin
    for (i=0; i<MEM_WORDS; i=i+1) mem[i]=32'd0;
    if (MEM_HEX != "") $readmemh(MEM_HEX, mem);
end

```

- 32-bit word 배열 하나가 명령어와 데이터를 함께 담습니다(통합 메모리). 코어 쪽 인터페이스는 imem/dmem 두 포트로 나뉘어 있지만 저장소는 하나이므로, 프로그램이 자기 코드 영역에 sw 를 하면 코드가 바뀝니다(self-modifying code 가능).
- initial 에서 먼저 0으로 채우고 MEM_HEX 가 지정되면 \$readmemh 로 덮어씁니다. HEX 파일에 있는 word 수만큼만 채워지고 나머지는 0으로 남습니다. \$readmemh 의 파일 경로는 시뮬레이터/Vivado를 실행한 현재 디렉토리 기준입니다. "firmware/nexys_hello.hex" 라는 상대 경로가 동작하려면 프로젝트 루트에서 실행해야 하며, 모든 스크립트가 cd "\$PROJECT_DIR" 을 먼저 하는 이유입니다.
- FPGA에서 initial 블록은 configuration 시점의 초기값으로만 반영됩니다. CPU_RESETN 버튼은 메모리를 다시 초기화하지 않습니다. 프로그램이 실행 중 바꾼 RAM 내용은 reset 후에도 남습니다(9.5절에서 .bss / .data 문제로 다시 언급).

4.6.2 주소 디코딩 (44~58행)

```

localparam MEM_BYTES = MEM_WORDS * 4;
wire imem_sel = (iaddr < MEM_BYTES);
wire dmem_sel = (daddr < MEM_BYTES);
wire [31:0] irdata = imem_sel ? mem[iaddr[15:2]] : 32'h0000_0013;

```

```

wire timer_sel = (daddr[31:12] == 20'h10001);
wire timer_wr = timer_sel && (!dwstrb);

```

- `iaddr[15:2]` : 바이트 주소를 4로 나눈 word 인덱스입니다. 하위 2비트는 명령어 정렬 때문에 항상 0이라 버립니다.
- 범위 밖 fetch에 `0x13 (NOP)`을 돌려주어 X가 CPU로 퍼지는 것을 막습니다. 실제 시스템이라면 `access fault trap`이 맞지만 이 버전은 구현하지 않았습니다.
- `timer_sel` 은 `0x1000_1000 ~ 0x1000_1FFF` page 전체를 timer로 봅니다. page 안에서는 `addr[2]` 만 쓰므로 `0x1000_1008` 은 `0x1000_1000` 의 alias입니다.
- `timer_wr` 은 byte enable 중 하나라도 켜지면 참입니다. timer에는 byte strobe가 전달되지 않으므로 sb 를 해도 32비트 전체가 갱신됩니다. timer MMIO에는 정렬된 sw 만 쓰는 것이 이 구현의 계약입니다.

4.6.3 데이터 읽기 mux (67~72행)

```

always @* begin
    if (dmem_sel) drdata = mem[daddr[15:2]];
    else if (daddr == 32'h1000_0004) drdata = {31'd0, uart_tx_ready};
    else if (timer_sel) drdata = timer_rdata;
    else drdata = 32'd0;
end

```

RAM이 최우선, 그 다음 UART ready, timer, 나머지는 0입니다. `0x1000_0000` (UART 데이터 레지스터)을 읽으면 0이 나옵니다. 이 mux 전체가 조합 논리여서 load 명령이 같은 cycle에 완료됩니다.

4.6.4 쓰기 (76~93행)

```

always @(posedge clk) begin
    uart_tx_valid <= 1'b0;
    if (!rst) begin
        if ((daddr == 32'h1000_0000) && (!dwstrb) && uart_tx_ready) begin
            uart_tx_data <= dwd[7:0];
            uart_tx_valid <= 1'b1;
        end
        if (dmem_sel && (!dwstrb)) begin
            if (dwstrb[0]) mem[daddr[15:2]][7:0] <= dwd[7:0];
            if (dwstrb[1]) mem[daddr[15:2]][15:8] <= dwd[15:8];
            if (dwstrb[2]) mem[daddr[15:2]][23:16] <= dwd[23:16];
            if (dwstrb[3]) mem[daddr[15:2]][31:24] <= dwd[31:24];
        end
    end
end

```

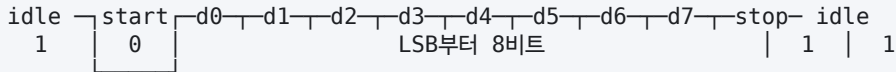
- `uart_tx_valid`는 첫 줄에서 매 cycle 0으로 놓고, UART 쓰기가 수락된 cycle에만 1로 덮어씹습니다. 결과적으로 정확히 한 cycle짜리 pulse가 됩니다.
- `uart_tx_ready`가 0이면 쓰기를 버립니다. 데이터 유실을 막는 책임은 소프트웨어에 있습니다: `0x1000_0004`를 polling해서 1일 때만 씁니다(`c_demo.c`의 `uart_putc`, `nexys_hello.S`의 2: 루프). 테스트벤치에서는 ready가 상수 1이라 항상 수락됩니다.
- RAM 쓰기는 lane별로 나뉘어 있어 sb / sh / sw가 모두 이 네 줄로 처리됩니다. 이 "부분 쓰기 가능한 배열 + 두 개의 조합식 읽기 포트" 조합 때문에 Vivado는 Block RAM이 아닌 LUT 기반 distributed RAM을 씁니다(8장).

4.7 rtl/uart_tx.v — 8-N-1 UART 송신기

4.7.1 UART 프로토콜 개념

UART는 clock 선 없이 한 가닥의 데이터 선으로 바이트를 보내는 직렬 통신입니다. 송수신 양쪽이 같은 속도(baud rate)를 미리 약속합니다. 115200 baud = 초당 115,200비트 = 비트당 약 8.68 μ s.

한 바이트(frame)는 다음 순서로 보냅니다. 선은 쉴 때(idle) 1입니다.



"8-N-1" = 데이터 8비트, 패리티 없음(None), stop 비트 1개. 총 10비트이므로 바이트 하나에 약 86.8 μ s가 걸립니다. CPU 명령 하나가 80 ns이므로 바이트 하나 보내는 동안 CPU는 약 1,000개 명령을 실행할 수 있습니다. 그래서 ready polling이 필요합니다.

4.7.2 구현

```
localparam integer CLKS_PER_BIT = CLOCK_HZ / BAUD;    // 100_000_000 / 115_200 = 868
localparam integer COUNT_W = $clog2(CLKS_PER_BIT);    // 10비트 카운터
reg [COUNT_W-1:0] count;
reg [3:0] bit_index;
reg [9:0] shift;
reg busy;
assign ready = !busy;
```

868 clock마다 비트 하나를 내보냅니다. 실제 baud는 100 MHz / 868 $\approx$ 115,207로 오차 0.006%입니다(허용 오차는 보통 $\pm 2\sim 3\%$).

```
end else if (!busy) begin
    tx <= 1'b1;
    if (valid) begin
        shift <= {1'b1, data, 1'b0}; // {stop, d7..d0, start}
        count <= CLKS_PER_BIT - 1;
        bit_index <= 0;
        busy <= 1'b1;
        tx <= 1'b0; // start bit 즉시 출력
    end
end else if (count != 0) begin // 현재 비트 유지
    count <= count - 1'b1;
end else if (bit_index == 9) begin // stop bit 끝 → idle
    busy <= 1'b0;
    tx <= 1'b1;
end else begin
    bit_index <= bit_index + 1'b1;
    shift <= {1'b1, shift[9:1]}; // 오른쪽으로 한 칸, 위에는 1(idle) 채움
    tx <= shift[1]; // 다음 비트 출력
    count <= CLKS_PER_BIT - 1;
end
```

동작을 표로 따라가면:

상태	bit_index	이 전환에서 tx에 나가는 것
valid 수락	0	start(0)

상태	bit_index	이 전환에서 tx 에 나가는 것
count 만료 1회째	0→1	shift[1] = d0
2회째	1→2	d1
...	...	...
8회째	7→8	d7
9회째	8→9	stop(1)
10회째	9	busy=0, idle

shift 를 오른쪽으로 밀 때 위쪽을 1로 채우므로, 몇 번을 밀어도 뒤에는 idle 값이 따라옵니다. ready = !busy 이므로 전송 중(약 87 μ s)에는 SoC가 새 바이트를 받지 않습니다.

4.8 rtl/nexys_a7_top.v — 보드 top

4.8.1 clock 분주와 BUFG (30~49행)

```
reg [2:0] cpu_div;
always @(posedge CLK100MHZ or negedge CPU_RESETN) begin
    if (!CPU_RESETN) begin
        reset_sync <= 2'b11;
        cpu_div <= 3'd0;
    end else begin
        reset_sync <= {reset_sync[0], 1'b0};
        cpu_div <= cpu_div + 1'b1;
    end
end
wire rst_100 = reset_sync[1];
BUFG cpu_clk_buf (.I(cpu_div[2]), .O(cpu_clk));
```

3비트 카운터가 100 MHz로 증가하면 bit 0은 50 MHz, bit 1은 25 MHz, bit 2는 12.5 MHz(주기 8 clock = 80 ns)의 정확한 사각파입니다. 이 신호는 일반 논리(fabric)에서 만들어졌으므로 그대로 clock으로 쓰면 skew가 큼니다. BUFG (Xilinx 전용 global clock buffer primitive)에 태워 칩 전체에 균일하게 배달합니다. XDC의 create_generated_clock ... [get_pins cpu_clk_buf/0] 가 바로 이 buffer 출력을 가리킵니다(6.8절).

더 정석적인 방법은 MMCM/PLL로 clock을 만드는 것이지만, 이 프로젝트는 벤더 IP 없이 카운터+BUFG로 해결했습니다. 분주비가 정확히 8이라 STA(정적 타이밍 분석)가 두 clock의 관계를 알 수 있습니다.

4.8.2 Reset 동기화 (28~58행)

CPU_RESETN은 버튼이라 아무 때나 바뀝니다. 각 clock domain마다 2단 shift register(reset_sync, cpu_reset_sync)를 두어 비동기로 assert(즉시 reset), 동기로 deassert(clock edge에 맞춰 해제) 합니다. 해제 순간이 clock edge와 겹쳐 생기는 metastability를 막는 표준 기법입니다. always @(posedge clk or negedge CPU_RESETN) 형태가 그 의도를 나타냅니다. 결과 rst_100, cpu_rst는 active-high로 각 모듈에 들어갑니다.

4.8.3 SoC와 UART 연결, CDC (60~79행)

```
rv32_soc #(.MEM_WORDS(2048), .MEM_HEX("firmware/nexys_hello.hex")) soc (
    .clk(cpu_clk), .rst(cpu_rst), .uart_tx_ready(uart_ready),
    .uart_tx_data(uart_data), .uart_tx_valid(uart_valid), .debug_pc(debug_pc)
```

```
);
uart_tx #(.CLOCK_HZ(100_000_000), .BAUD(115_200)) serial (
    .clk(CLK100MHZ), .rst(rst_100), .valid(uart_valid), .data(uart_data),
    .ready(uart_ready), .tx(UART_RXD_OUT)
);
```

uart_valid / uart_data 는 12.5 MHz에서 만들어져 100 MHz 모듈로 들어갑니다(clock domain crossing, CDC). 일반적으로 CDC에는 synchronizer가 필요하지만 여기서는 다음 이유로 직접 연결이 안전합니다.

1. 두 clock은 같은 원천에서 정확히 8분주된 **관계 있는(related) clock**이며, XDC에 generated clock으로 선언되어 Vivado가 두 domain 사이 경로를 **정상적으로 timing 분석**합니다(비동기 clock처럼 무시하지 않음).
2. uart_valid 는 CPU cycle 하나(80 ns) 동안 유지되므로 100 MHz 쪽은 이를 8번의 edge에서 봅니다. 첫 edge에서 busy=1 이 되므로 중복 수락은 없습니다.
3. 반대 방향의 ready 는 level 신호이고, SoC는 이를 edge에서 sample만 합니다.

FPGA 핀 이름 UART_RXD_OUT 은 "USB-UART 브리지 칩 입장에서 RXD"라는 뜻이라 FPGA에서는 출력입니다.

4.8.4 LED (83행)

```
assign LED = debug_pc[17:2];
```

PC의 bit [17:2]를 LED 16개에 냅니다. 프로그램이 짧은 루프를 돌면 LED는 평균 밝기로 보이고, 무한 루프(j 3b)에 멈추면 그 주소가 고정으로 표시됩니다. 8 KiB 메모리에서는 PC가 0x1FFF 이하이므로 LED[10:0]만 변합니다.

4.9 한 명령어가 한 사이클에 흐르는 경로 추적

c_demo 의 반복문 본체 다섯 명령이 어떻게 처리되는지 추적합니다(build/firmware/c_demo.dis 기준). 시뮬레이션 clock 주기는 10 ns이고, 각 명령은 rising edge 사이의 한 구간에서 전부 계산되어 다음 edge에 commit됩니다.

```
18: 00e787b3 add x15,x15,x14 ; sum += i
1c: 00170713 addi x14,x14,1 ; i++
20: 06868613 addi x12,x13,104 ; x12 = &sum_limit (0x68)
24: 00062603 lw x12,0(x12) ; x12 = sum_limit (volatile이라 매번 다시 읽음)
28: fee678e3 bgeu x12,x14,18 ; if (sum_limit >= i) goto 18
```

PC = 0x18, add x15,x15,x14 (R형식, 조합 경로):

1. imem_addr = 0x18 → SoC mem[6] → insn = 0x00e787b3 (조합, 즉시)
2. opcode = 0110011, funct3 = 000, funct7 = 0000000, rs1 = 15, rs2 = 14, rd = 15
3. 레지스터 파일이 regs[15], regs[14] 를 즉시 출력
4. 디코더: OP → alu_op = ALU_ADD, alu_a = rs1_data, alu_b = rs2_data, rd_we = 1, rd_data = alu_y
5. next_pc = 0x1c, dwstrb = 0000
6. rising edge: regs[15] <= 합, pc <= 0x1c

PC = 0x24, lw x12,0(x12) (I형식, 가장 긴 경로):

1. fetch → insn = 0x00062603, opcode = 0000011, funct3 = 010 (LW), rs1 = 12, rd = 12

2. `daddr_r = regs[12] + 0 = 0x68 → dmem_addr`
3. SoC: `dmem_sel = 1 → drdata = mem[0x1A] = 0x0000000a` (조합)
4. core: `rd_data = dmem_rdata = 10, rd_we = 1`
5. **rising edge**: `regs[12] <= 10, pc <= 0x28`

이 경로(명령어 메모리 → 디코드 → 레지스터 읽기 → 덧셈 → 데이터 메모리 → mux → 레지스터 쓰기 setup)가 **critical path**이며, 그 지연이 clock 주기의 하한을 정합니다.

PC = 0x28, bgeu x12,x14,18 (B형식):

1. `imm_b(0xfe678e3)` : bit31=1(부호), bit7=1, bits[30:25]=111111, bits[11:8]=1000 → `1_1111_0000` → `-16 = 0xFFFFFFF0`
2. `rs1_data(10) >= rs2_data(i)` unsigned 비교
3. 참이면 `next_pc = 0x28 + (-16) = 0x18`, 거짓이면 `0x2c`
4. **rising edge**: `pc <= next_pc`. 레지스터/메모리 쓰기 없음

`i`가 1부터 시작해 `sum_limit(10) >= i`가 성립하는 동안 10번 반복하므로 `sum = 55`가 되고, `0x2c`의 `sw x15,1024(x0)`가 이를 주소 `0x400`에 기록합니다. 테스트벤치는 이 `word(mem[0x100])`를 검사합니다.

5. 펌웨어(소프트웨어) 상세 설명

5.1 개념: 소스 코드가 메모리 초기값이 되기까지

```
c_demo.c —gcc -S—> c_demo.s —gcc(as+ld)—> c_demo.elf —objcopy—> c_demo.bin
—bin2hex.py—> c_demo.hex
(C 소스)      (어셈블리)      linker.ld      (ELF 실행파일)      (순수 바이트열)
($readmemh 텍스트)

└─objdump -d—> c_demo.dis (역어셈블:
검증용)
```

각 단계에서 나오는 파일의 성격:

파일	형식	내용
.c / .S	텍스트	사람이 쓴 소스. .S (대문자)는 C 전처리기를 거치는 어셈블리
.s	텍스트	컴파일러가 생성한 어셈블리. 어떤 명령어를 골랐는지 볼 수 있음
.o	ELF object	아직 주소가 확정되지 않은 기계어
.elf	ELF 실행파일	주소가 확정된 기계어 + 심볼 + 섹션 정보. 디버거/objdump가 읽음
.bin	raw binary	메모리 주소 0부터의 바이트를 그대로 나열. 헤더 없음
.hex	텍스트	.bin 을 32-bit little-endian word 단위로 16진수 한 줄씩. Verilog \$readmemh 형식
.dis	텍스트	.elf 를 역어셈블한 결과. 주소·기계어·니모닉이 함께 보여 검증에 최적
.map	텍스트	링커가 각 섹션/심볼을 어디에 두었는지 기록

Cross compile: 호스트(x86-64 Linux)에서 다른 아키텍처(RISC-V)용 코드를 만드는 것입니다. 도구 이름 앞의 riscv64-unknown-elf-가 target triple입니다. riscv64 라도 -march=rv32i -mabi=ilp32 를 주면 32-bit 코드를 만듭니다. elf 는 "OS 없음(bare-metal)"을 뜻하는 관례입니다.

Freestanding(독립 실행) 환경: OS도 C 표준 라이브러리도 없습니다. main 전에 실행되는 시작 코드(crt0)도 없으므로, 프로그램이 직접 reset 벡터(주소 0)에 놓일 _start 를 제공해야 합니다. printf 도 없어서 UART에 직접 바이트를 씁니다.

5.2 firmware/linker.ld — 링커 스크립트

링커 스크립트는 "어떤 코드/데이터를 메모리 어디에 놓을지"를 정합니다.

```
OUTPUT_ARCH(riscv)
ENTRY(_start)
```

출력 아키텍처와 진입점 심볼입니다. ENTRY 는 ELF 헤더의 entry 필드에 기록될 뿐, 실제 CPU는 항상 주소 0에서 시작합니다. 그래서 _start 를 0번지에 두는 것은 아래 SECTIONS 가 담당합니다.

```
MEMORY
{
```

```
RAM (rwx) : ORIGIN = 0x00000000, LENGTH = 8K
}
```

메모리 영역 하나: 주소 0부터 8 KiB, 읽기/쓰기/실행 모두 가능. nexys_a7_top.v 의 MEM_WORDS(2048) 와 같은 크기입니다. 프로그램이 8 KiB를 넘으면 링커가 "region RAM overflowed" 오류를 냅니다.

```
SECTIONS
{
    . = ORIGIN(RAM);

    .text : ALIGN(4)
    {
        KEEP(*( .text.start))
        *(.text .text.*)
        *(.rodata .rodata.*)
    } > RAM
```

- . 은 "현재 위치 카운터". 0에서 시작합니다.
- .text 출력 섹션에 먼저 .text.start 입력 섹션을 넣습니다. c_demo.c 의 `__attribute__((section(".text.start")))` 와 nexys_hello.S 의 `.section .text.start` 가 `_start` 를 이 섹션에 넣으므로, `_start` 가 반드시 주소 0이 됩니다. KEEP 은 참조가 없어도 링커가 버리지 못하게 합니다.
- 이어서 일반 코드(.text*)와 읽기 전용 데이터(.rodata*, 문자열 상수 등)를 붙입니다. c_demo.map 을 보면 .text.start 0x00~0x5F(96바이트), .rodata ("C OK\n") 0x60~0x65입니다.

```
.data : ALIGN(4)
{
    *(.data .data.*)
} > RAM

.bss (NOLOAD) : ALIGN(4)
{
    *(.bss .bss.*)
    *(COMMON)
} > RAM
```

- .data: 초기값이 있는 전역 변수. c_demo 의 `sum_limit = 10` 이 0x68에 놓입니다. 보통의 시스템은 .data 를 ROM에 두고 시작 코드가 RAM으로 복사하지만, 여기서는 HEX 이미지 전체가 RAM 초기값이므로 복사가 필요 없습니다.
- .bss: 초기값 0인 전역 변수. NOLOAD 라 파일에는 공간을 차지하지 않습니다. rv32_soc 가 메모리를 0으로 초기화하므로 첫 부팅에서는 0이 보장되지만, reset 버튼 후에는 이전 실행이 남긴 값이 남습니다(9.5절).

```
/DISCARD/ :
{
    *(.comment)
    *(.riscv.attributes)
}
}
```

컴파일러 버전 문자열과 RISC-V 속성 섹션을 버려 .bin 이 불필요하게 커지지 않게 합니다. .bin 은 "가장 낮은 섹션부터 가장 높은 섹션까지 연속"으로 만들어지므로, 이상한 주소의 섹션이 하나라도 있으면 파일이 폭발적으로 커집니다.

스택과 **.bss** 심볼: `_bss_start / _bss_end` (.bss 경계)와 `_stack_top` (RAM 끝 0x2000)은 mini OS의 boot.S 가 씁니다(9.6절). `c_demo` 는 -O1 에서 모든 함수가 inline되어 스택을 전혀 쓰지 않으므로 이 심볼을 참조하지 않지만, 함수 호출이 있는 프로그램에서는 `sp` 초기화가 반드시 필요합니다.

5.3 firmware/nexys_hello.S — 어셈블리 데모 (FPGA 기본 이미지)

```
.option norvc
.section .text.start, "ax", @progbits
.globl _start
_start:
    lui ra, 0x10000          /* UART base: 0x10000000 */
    li sp, 0x100             /* message address */
1:    lbu gp, 0(sp)
    beqz gp, 3f
2:    lw tp, 4(ra)           /* UART ready status */
    beqz tp, 2b
    sb gp, 0(ra)
    addi sp, sp, 1
    j 1b
3:    j 3b

.org 0x100
message:
.asciz "Hello Nexys A7!\r\n"
```

한 줄씩:

줄	의미
<code>.option norvc</code>	압축(16-bit) 명령어 생성 금지. CPU가 C 확장을 지원하지 않음
<code>.section .text.start, "ax", @progbits</code>	링커 스크립트가 0번지에 놓는 섹션. a =할당, x =실행 가능
<code>lui ra, 0x10000</code>	<code>ra(x1) = 0x1000_0000</code> . UART base 주소. 여기서 <code>ra</code> 는 그냥 임시 레지스터로 쓰임
<code>li sp, 0x100</code>	<code>sp(x2) = 0x100</code> . 문자열 포인터. <code>li</code> 는 pseudo-instruction으로 <code>addi x2, x0, 0x100</code> 으로 어셈블됨
<code>1: lbu gp, 0(sp)</code>	문자 한 바이트를 0 확장하여 <code>gp(x3)</code> 에 로드
<code>beqz gp, 3f</code>	0(문자열 끝)이면 앞쪽(forward) 라벨 3으로. <code>beqz</code> 는 <code>beq gp, x0</code> 의 pseudo
<code>2: lw tp, 4(ra)</code>	<code>0x1000_0004</code> (UART ready) 읽기
<code>beqz tp, 2b</code>	ready가 0이면 뒤쪽(backward) 라벨 2로 → busy-wait polling
<code>sb gp, 0(ra)</code>	<code>0x1000_0000</code> 에 바이트 쓰기 → UART 송신 시작
<code>addi sp, sp, 1</code>	다음 문자
<code>j 1b</code>	루프. <code>jal x0, 1b</code>
<code>3: j 3b</code>	끝. 자기 자신으로 무한 점프. bare-metal 프로그램은 "return"할 곳이 없음
<code>.org 0x100</code>	위치 카운터를 0x100으로 옮김(사이는 0으로 채움)
<code>.asciz "..."</code>	NUL 종료 문자열. <code>\r\n</code> 은 터미널에서 줄 처음으로 이동 + 줄바꿈

숫자 라벨 1:, 2:, 3: 은 GNU 어셈블러의 지역 라벨이며 1b (backward, 뒤로 가장 가까운 1), 3f (forward)로 참조합니다.

HEX와 대조: firmware/nexys_hello.hex 의 처음 10줄이 위 10개 명령어입니다.

주소	HEX	명령어	검산
0x00	100000b7	lui x1, 0x10000	imm[31:12]=0x10000, rd=00001, opcode=0110111
0x04	10000113	addi x2, x0, 0x100	imm=0x100, rs1=0, funct3=000, rd=00010, opcode=0010011
0x08	00014183	lbu x3, 0(x2)	funct3=100(LBU), rs1=2, rd=3, opcode=0000011
0x0C	00018c63	beq x3, x0, +24	0x0C+24 = 0x24 (라벨 3)
0x10	0040a203	lw x4, 4(x1)	imm=4, funct3=010
0x14	fe020ee3	beq x4, x0, -4	0x14-4 = 0x10 (라벨 2)
0x18	00308023	sb x3, 0(x1)	S형식, funct3=000
0x1C	00110113	addi x2, x2, 1	
0x20	fe9ff06f	jal x0, -24	0x20-24 = 0x08 (라벨 1)
0x24	0000006f	jal x0, 0	자기 자신
0x28~0xFC	00000000	(채움)	.org 0x100 때문
0x100	6c6c6548	"Hell"	'H'=0x48이 최하위 바이트 (little-endian)
0x104	654e206f	"o Ne"	
0x108	20737978	"xys "	
0x10C	0d213741	"A7!\r"	
0x110	0000000a	"\n\0\0\0"	

이 이미지는 nexys_a7_top.v 가 합성 시 메모리 초기값으로 공급됩니다. FPGA 보드 전원을 켜면 UART 터미널 (115200-8-N-1)에 Hello Nexys A7! 가 한 번 출력되고 CPU는 0x24에서 멈춥니다(LED에 0x24>>2 = 9 = 0b1001 표시).

5.4 firmware/c_demo.c — freestanding C 데모

5.4.1 C 소스

```
typedef unsigned int uint32_t;
typedef unsigned char uint8_t;
```

표준 헤더(<stdint.h>)조차 쓰지 않고 직접 정의합니다. 이 toolchain에는 헤더가 있지만, "아무 의존성 없음"을 보여 주려는 의도입니다.

```
#define UART_TX_ADDR      0x10000000u
#define UART_READY_ADDR  0x10000004u
#define SIGNATURE_ADDR    0x00000400u

volatile uint32_t sum_limit = 10u;
```

`volatile`이 핵심입니다. 없으면 GCC가 $1+...+10 = 55$ 를 컴파일 시간에 계산해 루프를 통째로 없애 버립니다. `volatile`은 "이 변수는 메모리에서 매번 실제로 읽어라"는 지시이므로 루프가 살아남고, 실제 CPU의 `lw / add / bgeu`를 검증할 수 있습니다. `.dis`에서 루프마다 `lw x12,0(x12)`가 반복되는 이유입니다.

```
static void uart_putc(uint8_t value)
{
    volatile uint32_t *const uart_ready = (volatile uint32_t *)UART_READY_ADDR;
    volatile uint8_t *const uart_tx = (volatile uint8_t *)UART_TX_ADDR;

    while ((*uart_ready & 1u) == 0u) { }
    *uart_tx = value;
}
```

MMIO 접근의 C 관용구입니다. 정수 주소를 `volatile` 포인터로 캐스팅하고 역참조합니다. `volatile`이 없으면 컴파일러가 "같은 주소를 반복해서 읽는 루프"를 한 번 읽는 것으로 최적화해 무한 루프가 됩니다. `uart_tx`가 `uint8_t*`이므로 컴파일러는 `sb`를 생성합니다.

```
__attribute__((section(".text.start"), noreturn))
void _start(void)
{
    volatile uint32_t *const signature = (volatile uint32_t *)SIGNATURE_ADDR;
    static const char message[] = "C OK\n";
    uint32_t sum = 0u;
    uint32_t i;

    for (i = 1u; i <= sum_limit; ++i)
        sum += i;

    *signature = sum;

    for (i = 0u; message[i] != '\0'; ++i)
        uart_putc((uint8_t)message[i]);

    for (;;) { }
}
```

- `section(".text.start")`: 이 함수를 링커 스크립트가 0번지에 놓는 섹션에 넣습니다. 즉 `_start`의 첫 명령이 `reset` 후 첫 명령입니다.
- `noreturn`: 반환하지 않는다고 알려 컴파일러가 반환 코드(`ret`)와 스택 프레임을 만들지 않게 합니다. 스택 포인터가 초기화되지 않았으므로 스택을 쓰지 않는 것이 중요합니다.
- `static const char message[]`: `.rodata`에 놓입니다(0x60).
- 결과 55를 0x400에 씁니다. 테스트벤치가 이 "서명(signature)"을 검사합니다.
- 마지막 무한 루프는 bare-metal 프로그램의 관례입니다.

5.4.2 컴파일러가 만든 어셈블리 (build/firmware/c_demo.s)

`make c-demo`가 `-S`로 생성합니다. 핵심 부분:

```
_start:
    lui a5,%hi(.LANCHOR0)      # sum_limit의 상위 20비트 (=0)
    lw a5,%lo(.LANCHOR0)(a5)   # a5 = sum_limit
    beqz a5,.L2                # sum_limit == 0이면 루프 건너뛰
    li a4,1                    # i = 1
    li a5,0                    # sum = 0
    lui a3,%hi(.LANCHOR0)
.L3:
```

```

add a5,a5,a4          # sum += i
addi a4,a4,1          # i++
addi a2,a3,%lo(.LANCHOR0)
lw a2,0(a2)           # sum_limit 다시 읽기 (volatile)
bgeu a2,a4,.L3        # sum_limit >= i 이면 반복
.L2:
sw a5,1024(zero)      # *(0x400) = sum
lui a3,%hi(.LANCHOR1)
addi a3,a3,%lo(.LANCHOR1) # a3 = message
li a2,67              # a2 = 'C' (첫 글자를 미리 로드)
li a4,268435456       # a4 = 0x100000000
.L4:
lw a5,4(a4)           # ready 읽기
andi a5,a5,1
beqz a5,.L4           # ready == 0 이면 대기
sb a2,0(a4)           # UART에 쓰기
addi a3,a3,1
lbu a2,0(a3)          # 다음 문자
bnez a2,.L4           # NUL이 아니면 반복
.L6:
j .L6

```

관찰 포인트:

- `%hi(sym) / %lo(sym)` : 32-bit 주소를 `lui` (상위 20비트) + `addi / lw offset` (하위 12비트)으로 나눠 만드는 relocation입니다. 주소가 `0x68`이라 `%hi`는 0이지만 컴파일러는 일반적인 코드를 그대로 냅니다.
- `uart_putc`가 사라졌습니다. -O1에서 `static` 함수가 호출자 하나뿐이면 inline됩니다. 그래서 `call / ret`도, 스택도 없습니다.
- `-msmall-data-limit=0` 덕분에 `sum_limit`이 `.sdata`가 아닌 `.data`에 있고 `gp` 기준 접근이 아닌 절대 주소 접근이 되었습니다. `gp`를 초기화하는 코드가 없으므로 이 옵션이 없으면 잘못된 주소를 읽게 됩니다(6.2절).
- `sw a5,1024(zero)` : `x0 + 1024 = 0x400`. base 레지스터로 `zero`를 쓰면 주소 로드 없이 0~2047 범위에 접근할 수 있습니다.
- `.attribute arch, "rv32i2p0"` : 이 코드가 RV32I 2.0만 필요함을 기록합니다.

5.4.3 역어셈블과 HEX 대조 (build/firmware/c_demo.dis, firmware/c_demo.hex)

```

00000000 <_start>:
0: 000007b7      lui x15,0x0
4: 0687a783      lw x15,104(x15)      # 0x68 = sum_limit
8: 02078263      beq x15,x0,2c
c: 00100713      addi x14,x0,1
10: 00000793      addi x15,x0,0
14: 000006b7      lui x13,0x0
18: 00e787b3      add x15,x15,x14
1c: 00170713      addi x14,x14,1
20: 06868613      addi x12,x13,104
24: 00062603      lw x12,0(x12)
28: fee678e3      bgeu x12,x14,18
2c: 40f02023      sw x15,1024(x0)     # 0x400 = signature
30: 000006b7      lui x13,0x0
34: 06068693      addi x13,x13,96     # 0x60 = message
38: 04300613      addi x12,x0,67      # 'C'
3c: 10000737      lui x14,0x10000
40: 00472783      lw x15,4(x14)       # UART ready
44: 0017f793      andi x15,x15,1
48: fe078ce3      beq x15,x0,40
4c: 00c70023      sb x12,0(x14)       # UART TX
50: 00168693      addi x13,x13,1
54: 0006c603      lbu x12,0(x13)
58: fe0614e3      bne x12,x0,40
5c: 0000006f      jal x0,5c

```

```
00000060 <message.952>:
60: 4b4f2043          "C OK"
```

.dis 의 가운데 열(기계어)이 c_demo.hex 의 각 줄과 1:1로 같습니다. 24개 명령(0x00~0x5C) + "C OK" (0x60) + "\n\0" (0x64, 0000000a) + sum_limit (0x68, 0000000a) = 27 word가 HEX의 27줄입니다. .bin 이 108바이트(=27×4)인 것도 일치합니다.

.dis 는 -M no-aliases,numeric 으로 만들어져 pseudo-instruction(li, beqz)이 아닌 실제 명령어와 레지스터 번호가 보입니다. 하드웨어 디코더와 대조할 때 이 표기가 편합니다.

5.5 HEX 파일 형식과 scripts/bin2hex.py

\$readmemh 가 읽는 형식은 아주 단순합니다: 한 줄에 word 하나, 16진수, 첫 줄이 mem[0] . 주소나 체크섬이 없습니다 (Intel HEX와 다름).

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("input", type=Path, help="raw input binary")
    parser.add_argument("output", type=Path, help="$readmemh output file")
    parser.add_argument(
        "--max-bytes",
        type=lambda value: int(value, 0),
        default=8192,
        help="fail if the image exceeds this size (default: 8192)",
    )
    args = parser.parse_args()
```

- int(value, 0) : 밑(base) 0은 접두사로 진법을 판단합니다. 8192, 0x2000 모두 허용됩니다.

```
data = args.input.read_bytes()
if len(data) > args.max_bytes:
    raise SystemExit(f"image is {len(data)} bytes, larger than {args.max_bytes}-byte memory")
```

메모리보다 큰 이미지는 여기서 실패시킵니다. 링커의 LENGTH = 8K 와 이중 안전장치입니다. SystemExit 에 문자열을 주면 메시지를 stderr에 찍고 종료 코드 1로 끝나므로 make 가 실패를 감지합니다.

```
data += bytes((-len(data)) % 4)
```

4의 배수가 되도록 0 바이트를 덧붙입니다. (-n) % 4 는 n=1→3, 2→2, 3→1, 0→0으로 "4의 배수까지 부족한 수"를 한 식으로 계산하는 관용구입니다.

```
words = (
    int.from_bytes(data[offset : offset + 4], "little")
    for offset in range(0, len(data), 4)
)
args.output.write_text("".join(f"{word:08x}\n" for word in words), encoding="ascii")
```

4바이트씩 잘라 little-endian 정수로 해석하고 8자리 16진수로 출력합니다. 08x 는 "0으로 채운 8자리 소문자 16진수". 바이트 43 20 4F 4B 가 4b4f2043 이 되는 곳이 바로 "little" 입니다. \$readmemh 로 읽은 word를 CPU가 lbu 로 바이트 0을 읽으면 다시 0x430이 나오므로, little-endian 규약이 toolchain → 스크립트 → 하드웨어까지 일관됩니다.

6. 빌드 시스템과 스크립트 상세 설명

6.1 Makefile 읽는 법 (기초)

make 타겟은 "타겟 파일이 없거나 의존 파일보다 오래되었으면 레시피를 실행"합니다.

타겟: 의존1 의존2 | 순서만-의존
레시피 (반드시 TAB으로 시작)

이 Makefile에서 쓰는 문법:

문법	의미
<code>VAR := 값</code>	즉시 확장 변수 (정의 시점에 값 확정)
<code>VAR ?= 값</code>	환경/명령줄에서 주지 않았을 때만 기본값. <code>make CROSS=riscv32-unknown-elf</code> 처럼 덮어쓸 수 있음
<code>\$(VAR)</code>	변수 참조
<code>\$@</code>	현재 타겟 이름
<code>\$<</code>	첫 번째 의존 파일
<code>\ </code> 뒤의 의존	order-only prerequisite: 존재만 보장하고 timestamp는 비교하지 않음. 디렉토리에 씀
<code>.PHONY</code>	파일이 아닌 "동작" 타겟. 같은 이름 파일이 있어도 항상 실행
<code>-Wl,a,b</code>	gcc가 링커(ld)에 a b를 전달

6.2 변수 정의

```
RTL := rtl/rv32_alu.v rtl/rv32_regfile.v rtl/rv32_csr.v rtl/rv32_core.v rtl/simple_timer.v  
rtl/rv32_soc.v
```

시뮬레이션/lint에 쓰는 RTL 목록입니다. `uart_tx.v`와 `nexys_a7_top.v`는 없습니다. BUFG는 Xilinx primitive라 Icarus/Verilator가 모르고, 테스트벤치는 `rv32_soc`만 쓰기 때문입니다. Vivado 스크립트는 `glob rtl/*.v`로 전부 읽습니다.

```
CROSS ?= riscv64-unknown-elf  
RISCV_GCC := $(CROSS)-gcc  
RISCV_OBJCOPY := $(CROSS)-objcopy  
RISCV_OBJDUMP := $(CROSS)-objdump  
PYTHON ?= python3
```

toolchain 접두사. 설치된 toolchain 이름이 다르면 `make CROSS=...`로 바꿉니다. 이 시스템에는 `/usr/bin/riscv64-unknown-elf-gcc` (GCC 9.3.0)가 있습니다.

```
FW_NAME := nexys_hello  
FW_BUILD := build/firmware  
FW_ELF := $(FW_BUILD)/$(FW_NAME).elf  
FW_BIN := $(FW_BUILD)/$(FW_NAME).bin  
FW_HEX := firmware/$(FW_NAME).hex
```

중간 파일은 build/firmware/ 에, 최종 HEX는 firmware/ 에 둡니다. HEX는 Vivado가 합성 시 읽는 입력이고 make clean 으로 지워지면 안 되며, toolchain이 없는 환경에서도 시뮬레이션/합성이 되도록 저장소에 포함하는 것입니다.

```
CFLAGS_RV32I := -march=rv32i -mabi=ilp32 -O1 -ffreestanding -fno-builtin \
-fno-pic -fno-asynchronous-unwind-tables -msmall-data-limit=0
```

C 컴파일 옵션 하나하나가 이 CPU의 제약과 대응합니다:

옵션	이유
-march=rv32i	RV32I 기본 명령만 생성. mul (M), 압축 명령(C) 등을 쓰면 CPU가 illegal instruction
-mabi=ilp32	int/long/pointer 모두 32-bit, 부동소수점 인자는 정수 레지스터로 전달 (F 확장 없음)
-O1	적당한 최적화. -O0 은 스택을 쓰는 코드를 만들어 sp 초기화가 필요해지고, -O2 는 검증하기 어려운 변환이 늘어남
-ffreestanding	표준 라이브러리/ main 의미론을 가정하지 않음
-fno-builtin	memcpy 등을 컴파일러가 마음대로 삽입/치환하지 않음 (라이브러리가 없으므로)
-fno-pic	위치 독립 코드 금지. GOT 없이 절대 주소 사용
-fno-asynchronous-unwind-tables	예외 처리용 .eh_frame 섹션 생성 안 함 → 바이너리 크기 절약
-msmall-data-limit=0	작은 전역 변수를 gp 상대 주소(.sdata)로 접근하지 않음. gp 를 초기화하는 코드가 없으므로 필수

6.3 펌웨어 타킷

```
.PHONY: all firmware firmware-disasm c-demo c-demo-disasm mini-os mini-os-disasm sim sim-c-demo
sim-mini-os lint vivado clean

all: firmware
firmware: $(FW_HEX)

$(FW_BUILD):
    mkdir -p $@
```

make 만 치면 nexys_hello.hex 를 만듭니다. \$(FW_BUILD) 규칙은 디렉토리를 만들며, 아래에서 order-only 의존으로 씁니다.

```
$(FW_ELF): firmware/$(FW_NAME).S firmware/linker.ld | $(FW_BUILD)
    $(RISCV_GCC) -march=rv32i -mabi=ilp32 -mno-relax \
        -nostdlib -nostartfiles -Wl,--build-id=none,--no-relax \
        -T firmware/linker.ld -o $@ $<
```

어셈블리 → ELF를 한 번에 합니다(gcc 가 as 와 ld 를 호출).

옵션	이유
-mno-relax, -Wl,--no-relax	linker relaxation(lui+addi 를 gp 상대 접근으로 줄이는 최적화) 금지. gp 가 없으므로
-nostdlib	libc, libgcc를 링크하지 않음

옵션	이유
<code>-nostartfiles</code>	<code>crt0</code> 등 시작 파일을 링크하지 않음. <code>_start</code> 는 우리가 제공
<code>-Wl,--build-id=none</code>	<code>.note.gnu.build-id</code> 섹션 생성 안 함. 링커 스크립트에 없는 섹션이 생겨 <code>.bin</code> 이 이상해지는 것을 방지
<code>-T firmware/linker.ld</code>	기본 링커 스크립트 대신 우리 것 사용

```
$(FW_BIN): $(FW_ELF)
    $(RISCV_OBJCOPY) -O binary $< $@

$(FW_HEX): $(FW_BIN) scripts/bin2hex.py
    $(PYTHON) scripts/bin2hex.py --max-bytes 8192 $< $@
```

`objcopy -O binary` 는 ELF의 로드 가능 섹션을 주소 순서대로 이어 붙인 순수 바이트열을 만듭니다. 이어서 5.5절의 스크립트로 HEX를 만듭니다. `scripts/bin2hex.py` 가 의존에 들어 있어 스크립트를 고치면 HEX가 다시 생성됩니다.

```
firmware-disasm: $(FW_ELF)
    $(RISCV_OBJDUMP) -d -M no-aliases,numeric $(FW_ELF)
```

역어셈블을 화면에 출력합니다. `-d` 는 코드 섹션 역어셈블, `-M no-aliases` 는 pseudo-instruction 대신 실제 명령어, `numeric` 은 ABI 이름 대신 x 번호.

6.4 C 데모 타킷

```
c-demo: $(C_DEMO_ASM) $(C_DEMO_HEX) $(C_DEMO_DIS)

$(C_DEMO_ASM): firmware/$(C_DEMO_NAME).c | $(FW_BUILD)
    $(RISCV_GCC) $(CFLAGS_RV32I) -S -o $@ $<
```

`-S` 는 컴파일러를 어셈블리 출력에서 멈춥니다. 이 `.s` 가 다음 단계의 입력이 되므로, "C → 어셈블리 → 기계어"의 중간 단계를 눈으로 볼 수 있습니다(교육용 의도).

```
$(C_DEMO_ELF): $(C_DEMO_ASM) firmware/linker.ld | $(FW_BUILD)
    $(RISCV_GCC) -march=rv32i -mabi=ilp32 -mno-relax \
        -nostdlib -nostartfiles -Wl,--build-id=none,--no-relax \
        -Wl,-Map=$(FW_BUILD)/$(C_DEMO_NAME).map \
        -T firmware/linker.ld -o $@ $<
```

`nexys_hello` 와 같은 링크 옵션에 `-Wl,-Map=...` 이 추가되어 맵 파일을 남깁니다. 맵 파일로 각 섹션의 주소와 크기를 확인할 수 있습니다(5.2절에서 인용).

```
$(C_DEMO_DIS): $(C_DEMO_ELF)
    $(RISCV_OBJDUMP) -d -S -M no-aliases,numeric $< > $@

c-demo-disasm: $(C_DEMO_DIS)
    cat $(C_DEMO_DIS)
```

`-S` (대문자)는 디버그 정보가 있으면 C 소스를 섞어 보여 줍니다(`-g` 없이 컴파일했으므로 여기서는 소스가 섞이지 않음).

6.5 시뮬레이션·lint·FPGA·정리 타킷

```
sim:
  mkdir -p build
  iverilog -g2012 -Wall -o build/tb_core $(RTL) tb/tb_rv32_core.v
  vvp build/tb_core
```

Icarus Verilog 두 단계: iverilog 가 컴파일하여 build/tb_core (vvp 바이트코드)를 만들고 vvp 가 실행합니다. -g2012 는 SystemVerilog-2012 문법 허용, -Wall 은 모든 경고.

```
sim-c-demo: c-demo
  bash scripts/run_c_demo_sim.sh
```

먼저 c-demo 로 HEX를 최신으로 만든 뒤 6.7절의 스크립트를 실행합니다.

```
lint:
  verilator --lint-only -Wall --top-module rv32_soc $(RTL)
```

Verilator를 정적 검사기로만 씁니다. latch, 폭 불일치, 미사용 신호 등을 잡습니다.

```
vivado: firmware
  vivado -mode batch -source scripts/build_nexys_a7.tcl
```

HEX가 최신인지 확인한 뒤 Vivado를 batch 모드로 실행합니다. 프로젝트 루트에서 실행되므로 Tcl 안의 상대 경로와 RTL의 "firmware/nexys_hello.hex" 가 맞습니다.

```
clean:
  rm -rf build
```

build/ 만 지웁니다. firmware/*.hex 는 남습니다.

6.6 명령어 요약 (무엇을 치면 무엇이 생기나)

명령	하는 일	생성물
make / make firmware	어셈블리 데모 빌드	firmware/nexys_hello.hex , build/firmware/nexys_hello.{elf,bin}
make firmware-disasm	위 ELF 역어셈블 출력	(화면)
make c-demo	C 데모 빌드	firmware/c_demo.hex , build/firmware/c_demo. {s,elf,bin,dis,map}
make c-demo-disasm	C 데모 역어셈블 출력	(화면)
make sim	smoke test 시뮬레이션	build/tb_core , "PASS: RV32I smoke test"
make sim-c-demo	C 데모 end-to-end 시뮬레이션	build/tb_c_demo , build/c_demo.vcd , "PASS: C firmware ..."

명령	하는 일	생성물
<code>make mini-os</code>	mini OS 빌드 (FPGA용 10 ms tick)	<code>firmware/mini_os.hex</code> , <code>build/firmware/mini_os.{elf,bin,dis,map}</code>
<code>make mini-os-disasm</code>	mini OS 역어셈블 출력	(화면)
<code>make sim-mini-os</code>	mini OS 시뮬레이션 (짧은 tick 이미지)	<code>build/firmware/mini_os_sim.hex</code> , <code>build/tb_mini_os</code> , <code>build/mini_os.vcd</code> , "PASS: mini OS boot..."
<code>make lint</code>	Verilator 정적 검사	(화면)
<code>make vivado</code>	합성·구현·bitstream	<code>build/vivado/nexys_a7_rv32i.bit</code> , <code>*.rpt</code> , <code>*.dcp</code>
<code>vivado -mode batch -source scripts/program_nexys_a7.tcl</code>	보드 프로그래밍	(보드)
<code>make clean</code>	생성물 삭제	—

6.7 scripts/run_c_demo_sim.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

-e: 명령 하나라도 실패하면 즉시 종료. -u: 정의 안 된 변수 사용 시 오류. -o pipefail: 파이프 중간 실패도 감지. 셸 스크립트의 안전 기본값입니다.

```
PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
LOCAL_ROOT="$PROJECT_DIR/build/tools/iverilog"
cd "$PROJECT_DIR"
mkdir -p build
```

스크립트 파일 위치(`scripts/`)의 상위를 프로젝트 루트로 삼고 그리로 이동합니다. 어디서 실행하든 `$readmemh("firmware/c_demo.hex")`와 `$dumpfile("build/c_demo.vcd")`의 상대 경로가 맞게 됩니다.

```
if command -v iverilog >/dev/null 2>&1 && command -v vvp >/dev/null 2>&1; then
    IVERILOG=(iverilog)
    VVP=(vvp)
elif [[ -x "$LOCAL_ROOT/usr/bin/iverilog" && -x "$LOCAL_ROOT/usr/bin/vvp" ]]; then
    IVL_LIB="$LOCAL_ROOT/usr/lib/x86_64-linux-gnu/ivl"
    IVERILOG=("$LOCAL_ROOT/usr/bin/iverilog" -B "$IVL_LIB")
    VVP=("$LOCAL_ROOT/usr/bin/vvp" -M "$IVL_LIB")
else
    echo "Icarus Verilog is required. Install it with: sudo apt install iverilog" >&2
    exit 1
fi
```

Icarus를 찾는 3단계 fallback입니다. (1) PATH에 있으면 그것, (2) 없으면 `build/tools/iverilog/` 아래에 패키지를 풀어 둔 로컬 사본(관리자 권한 없이 `.deb`를 `dpkg -x`로 풀면 이 구조가 됨). 이때 `-B` (iverilog에게 내부 도구 위치)와 `-M` (vvp에게 VPI 모듈 위치)로 라이브러리 경로를 알려 줘야 합니다. (3) 둘 다 없으면 설치 안내 후 실패. 배열 변수 `IVERILOG=(...)`와 `"${IVERILOG[@]}"`는 옵션이 포함된 명령을 안전하게 담는 bash 관용구입니다.

```
"${IVERILOG[@]}" -g2012 -Wall -o build/tb_c_demo \
    rtl/rv32_alu.v rtl/rv32_regfile.v rtl/rv32_csr.v rtl/rv32_core.v \
    rtl/simple_timer.v rtl/rv32_soc.v tb/tb_c_demo.v
"${VVP[@]}" build/tb_c_demo
```

컴파일 후 실행. 테스트벤치가 \$fatal 이면 vvp가 0이 아닌 코드로 끝나고 set -e 때문에 스크립트도 실패합니다.

6.8 constraints/nexys_a7_100t.xdc

XDC는 Vivado에 "물리적 사실"을 알려 주는 Tcl 명령 모음입니다.

```
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports CLK100MHZ]
create_clock -add -name sys_clk_pin -period 10.000 -waveform {0 5} [get_ports CLK100MHZ]
create_generated_clock -name cpu_clk -source [get_ports CLK100MHZ] -divide_by 8 [get_pins
cpu_clk_buf/0]
```

- 100 MHz 발진기는 핀 E3, 3.3 V CMOS 레벨.
- create_clock: 주기 10 ns, 0~5 ns가 high. 이 선언이 있어야 timing 분석이 시작됩니다.
- create_generated_clock: BUFG 출력이 100 MHz를 8로 나눈 clock임을 알립니다. 이것이 없으면 Vivado는 cpu_clk 을 clock으로 인식하지 못하거나 두 domain 사이 경로를 분석하지 않습니다. CDC 경로가 timing 검증되는 근거입니다(4.8.3절).

```
set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

configuration bank 전압 설정. 없으면 bitstream 생성 시 경고/오류.

```
set_property -dict { PACKAGE_PIN C12 IOSTANDARD LVCMOS33 } [get_ports CPU_RESETN]
set_property -dict { PACKAGE_PIN D4 IOSTANDARD LVCMOS33 } [get_ports UART_RXD_OUT]
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports {LED[0]}]
...
set_property -dict { PACKAGE_PIN V11 IOSTANDARD LVCMOS33 } [get_ports {LED[15]}]
```

Digilent가 공개한 Nexys A7 마스터 XDC에서 가져온 핀 번호입니다. 포트 이름은 nexys_a7_top.v 의 포트와 대소문자까지 일치해야 합니다.

6.9 scripts/build_nexys_a7.tcl — Vivado 비프로젝트 흐름

Vivado GUI 프로젝트(.xpr) 없이 Tcl로 전 과정을 돌리는 "non-project batch flow"입니다. 재현성이 좋고 git에 넣기 쉽습니다.

```
set root [file normalize [file join [file dirname [info script]] ..]]
set out [file join $root build vivado]
file mkdir $out
```

스크립트 위치 기준으로 프로젝트 루트와 출력 폴더를 구합니다.

```
read_verilog [glob [file join $root rtl *.v]]
```

```
read_xdc [file join $root constraints nexys_a7_100t.xdc]
```

rtl/*.v 전부(top, uart 포함)와 XDC를 읽습니다. \$readmemh 경로는 여기가 아니라 Vivado를 실행한 디렉토리 기준이므로 make vivado 처럼 루트에서 실행해야 합니다.

```
synth_design -top nexys_a7_top -part xc7a100tcs9324-1 -flatten_hierarchy rebuilt
write_checkpoint -force [file join $out post_synth.dcp]
report_utilization -file [file join $out utilization_synth.rpt]
```

- 합성: RTL → FPGA primitive(LUT, FF, distributed RAM, BUFG) netlist. 부품은 Nexys A7-100T의 Artix-7 100T, CSG324 패키지, speed grade -1.
- -flatten_hierarchy rebuilt: 최적화를 위해 모듈 경계를 허물고 나서 리포트용으로 계층 이름을 복원합니다.
- checkpoint(.dcp)는 이 시점의 상태 저장. 나중에 open_checkpoint 로 GUI에서 열어 볼 수 있습니다.

```
opt_design
place_design
phys_opt_design
route_design
write_checkpoint -force [file join $out post_route.dcp]
report_timing_summary -file [file join $out timing_summary.rpt]
report_utilization -file [file join $out utilization_route.rpt]
write_bitstream -force [file join $out nexys_a7_rv32i.bit]
```

구현(implementation) 단계: 논리 최적화 → 배치 → 배치 후 물리 최적화 → 배선 → 리포트 → bitstream. timing_summary.rpt 의 WNS/WHIS가 양수여야 설계가 지정 clock에서 안전합니다(8장).

6.10 scripts/program_nexys_a7.tcl — 보드 프로그래밍

```
set root [file normalize [file join [file dirname [info script]] ..]]
set bitfile [file join $root build vivado nexys_a7_rv32i.bit]

open_hw_manager
connect_hw_server
open_hw_target
set dev [lindex [get_hw_devices xc7a100t_0] 0]
if {$dev eq ""} { error "Nexys A7-100T (xc7a100t_0) not found" }
current_hw_device $dev
refresh_hw_device $dev
set_property PROGRAM.FILE $bitfile $dev
program_hw_devices $dev
```

Hardware Manager를 열고(open_hw_manager), 로컬 hw_server에 연결하고(connect_hw_server), JTAG 케이블(target)을 열고, 체인에서 xc7a100t_0 장치를 찾아 bitstream을 씁니다. 장치가 없으면 명확한 오류로 끝납니다. 이 방식은 SRAM에 쓰는 휘발성 프로그래밍이라 보드 전원을 끄면 사라집니다. 영구 저장은 QSPI flash에 별도 절차가 필요합니다.

6.11 scripts/run_xsim.tcl — Vivado 시뮬레이터용

```
run all
quit
```

Vivado 내장 시뮬레이터 xsim을 batch로 돌릴 때 -tclbatch로 넘기는 두 줄입니다. "\$finish"까지 실행하고 종료". build/xsim/ 과 build/c_demo_xsim/ 에 남아 있는 로그가 이 흐름을 썼던 흔적입니다. 사용 예:

```
# 프로젝트 루트에서
xvlog -sv rtl/rv32_alu.v rtl/rv32_regfile.v rtl/rv32_csr.v rtl/rv32_core.v \
      rtl/simple_timer.v rtl/rv32_soc.v tb/tb_c_demo.v
xelab -debug typical tb_c_demo -s c_demo_sim
xsim c_demo_sim -tclbatch scripts/run_xsim.tcl
```

(xvlog =컴파일, xelab =elaboration/링크, xsim =실행. 루트의 xvlog.log, xvlog.pb 는 xvlog가 남긴 로그입니다.)

6.12 start.sh / stop.sh — 슬라이드 서버

발표 자료(slides/index.html)를 로컬 HTTP 서버로 띄우는 보조 스크립트입니다. CPU와 직접 관계는 없지만 구조가 잘 짜여 있어 설명합니다.

start.sh

```
PID_FILE="$PROJECT_DIR/build/slides-server.pid"
LOG_FILE="$PROJECT_DIR/build/slides-server.log"

if [[ -f "$PID_FILE" ]]; then
    SERVER_PID="$(cat "$PID_FILE")"
    if kill -0 "$SERVER_PID" 2>/dev/null; then
        echo "Slide server is already running (PID $SERVER_PID)."
        exit 0
    fi
    rm -f "$PID_FILE"
fi
```

PID 파일로 중복 실행을 막습니다. kill -0 은 신호를 보내지 않고 "그 PID가 살아 있는가"만 검사합니다. 죽은 PID가 남아 있으면(stale) 파일을 지우고 계속합니다.

```
nohup python3 -m http.server 8000 --directory slides >"$LOG_FILE" 2>&1 &
SERVER_PID=$!
echo "$SERVER_PID" >"$PID_FILE"

sleep 0.5
if ! kill -0 "$SERVER_PID" 2>/dev/null; then
    echo "Failed to start slide server. See: $LOG_FILE" >&2
    rm -f "$PID_FILE"
    exit 1
fi
```

nohup ... &로 터미널을 닫아도 살아남게 백그라운드 실행하고, \$! (마지막 백그라운드 PID)를 기록합니다. 0.5초 뒤 살아 있는지 확인해 포트 충돌 같은 즉시 실패를 잡아냅니다.

stop.sh

```
COMMAND_LINE="$(tr '\0' ' ' <"/proc/$SERVER_PID/cmdline" 2>/dev/null || true)"
if [[ "$COMMAND_LINE" != *"python3 -m http.server 8000 --directory slides"* ]]; then
    echo "Refusing to stop PID $SERVER_PID: it is not the expected slide server." >&2
    exit 1
fi
```

PID 재사용으로 엉뚱한 프로세스를 죽이는 사고를 막습니다. /proc/PID/cmdline은 인자가 NUL로 구분되어 있어 tr로 공백으로 바꾼 뒤, 기대한 명령줄인지 확인합니다.

```
kill "$SERVER_PID"
for _ in {1..20}; do
    if ! kill -0 "$SERVER_PID" 2>/dev/null; then
        rm -f "$PID_FILE"; echo "Slide server stopped."; exit 0
    fi
    sleep 0.1
done
echo "Slide server did not stop within 2 seconds (PID $SERVER_PID)." >&2
exit 1
```

SIGTERM을 보내고 최대 2초(0.1초 × 20) 동안 종료를 기다립니다.

7. 시뮬레이션과 검증

7.1 검증의 층위

이 프로젝트는 서로 다른 네 가지 방법으로 설계를 확인합니다. 각각이 보장하는 것이 다르다는 점을 이해해야 결과를 과대 해석하지 않습니다.

방법	명령	확인하는 것	확인하지 못하는 것
Lint	<code>make lint</code>	문법, 폭 불일치, latch, 미사용 신호	기능 동작
Smoke test	<code>make sim</code>	12개 명령의 산술/메모리/분기/UART 경로	나머지 명령, CSR, trap
C end-to-end	<code>make sim-c-demo</code>	실제 toolchain 산출물이 CPU에서 옳게 실행됨	전체 ISA, interrupt, 장시간 동작
Mini OS end-to-end	<code>make sim-mini-os</code>	timer interrupt, ecall, trap 진입/복귀, context switch, 새 task 진입 (9.7절)	장시간 동작(timer wrap), 중첩/외부 interrupt
FPGA STA	<code>make vivado</code>	지정 clock에서 timing 충족, 자원 사용량	기능 동작(합성 결과가 RTL과 같다는 전제)

7.2 tb/tb_rv32_core.v — smoke test

```
reg clk=0, rst=1;
always #5 clk=~clk;           // 주기 10 ns
rv32_soc #(.MEM_WORDS(256)) dut ( ..., .uart_tx_ready(1'b1), ... );
```

메모리 256 word(1 KiB), UART는 항상 ready. MEM_HEX 를 주지 않고 테스트벤치가 계층 참조로 메모리에 기계어를 직접 씁니다:

```
dut.mem[0] = 32'h00500093; // addi x1,x0,5
dut.mem[1] = 32'h00700113; // addi x2,x0,7
dut.mem[2] = 32'h002081b3; // add x3,x1,x2 (=12)
dut.mem[3] = 32'h08302023; // sw x3,128(x0)
dut.mem[4] = 32'h08002203; // lw x4,128(x0)
dut.mem[5] = 32'h00320463; // beq x4,x3,+8
dut.mem[6] = 32'h00100293; // addi x5,x0,1 (skipped)
dut.mem[7] = 32'h02a00293; // addi x5,x0,42
dut.mem[8] = 32'h05500313; // addi x6,x0,'U'
dut.mem[9] = 32'h100003b7; // lui x7,0x10000
dut.mem[10] = 32'h00638023; // sb x6,0(x7) (UART)
dut.mem[11] = 32'h0000006f; // jal x0,0
```

프로그램의 의도: $x3 = 5 + 7 = 12$ 를 메모리 0x80에 저장했다가 다시 읽고, 같으면 $x5 = 1$ 을 건너뛰어 $x5 = 42$ 가 되게 한 뒤, 'U' (0x55)를 UART에 쓰고 멈춥니다. 세 검사가 각각 산술+store/load, 분기, MMIO 경로를 확인합니다.

```
#22 rst=0;
repeat (20) @(posedge clk);
if (dut.mem[32] !== 32'd12) begin $display("FAIL RAM: %h",dut.mem[32]); $fatal; end
if (dut.cpu.rf.regs[5] !== 32'd42) begin $display("FAIL BRANCH"); $fatal; end
if (uart_data !== 8'h55) begin $display("FAIL UART: %h",uart_data); $fatal; end
```

```
$display("PASS: RV32I smoke test");
$finish;
```

- #22 rst=0 : 22 ns에 reset 해제. 첫 edge(5 ns)와 세 번째 edge(25 ns) 사이라 reset이 최소 두 edge 동안 유지됩니다.
- 20 cycle이면 11개 명령을 다 돌고 무한 루프에 들어가기 충분합니다.
- dut.mem[32] 는 바이트 주소 0x80 ÷ 4.
- uart_data 는 SoC의 uart_tx_data 레지스터로, valid pulse 이후에도 값을 유지하므로 나중에 검사할 수 있습니다.

7.3 tb/tb_c_demo.v — C 데모 end-to-end

```
rv32_soc #(.MEM_WORDS(2048), .MEM_HEX("firmware/c_demo.hex")) dut ( ...
.uart_tx_ready(1'b1) ... );
```

FPGA와 같은 8 KiB, 실제 toolchain이 만든 HEX를 적재합니다.

```
wire [31:0] sum_x15 = dut.cpu.rf.regs[15];
wire [31:0] loop_i_x14 = dut.cpu.rf.regs[14];
wire      signature_write = (dut.cpu.dmem_addr == 32'h0000_0400) &&
(|dut.cpu.dmem_wstrb);
wire [31:0] signature_write_data = dut.cpu.dmem_wdata;
wire [31:0] signature_value = dut.mem[32'h400 >> 2];
```

파형에서 보기 좋게 내부 신호에 이름을 붙인 것입니다. 컴파일러가 sum을 x15(a5), i를 x14(a4)에 두었음을 5.4절에서 확인했으므로 이 두 레지스터를 뽑아 냅니다. 컴파일러 버전이 바뀌어 레지스터 할당이 달라지면 이 probe 이름은 틀릴 수 있지만 PASS/FAIL 판정에는 영향이 없습니다(판정은 메모리와 UART 바이트만 봅니다).

```
initial begin
    $dumpfile("build/c_demo.vcd");
    $dumpvars(0, tb_c_demo.dut.cpu);
    $dumpvars(0, tb_c_demo.clk);
    ...
end
```

VCD 파형을 만듭니다. dut.cpu 계층 전체(레지스터 파일, CSR 포함)를 덤프하되, 8 KiB mem 배열은 파일이 커지므로 제외합니다.

```
always @(posedge clk) begin
    if (uart_valid && received_count < 5) begin
        received[received_count] <= uart_data;
        received_count <= received_count + 1;
    end
end
```

UART 수신기 역할: valid pulse마다 바이트를 모읍니다.

```
#22 rst = 1'b0;
repeat (1000) @(posedge clk);
if (dut.mem[32'h400 >> 2] != 32'd55) begin $display("FAIL: signature=%0d, ...");
$fatal; end
```

```

    if (received_count != 5 || received[0] != "C" || ... || received[4] != 8'h0a) begin
... $fatal; end
    $display("PASS: C firmware signature=55, UART=\"C OK\\n\"");

```

1000 cycle 후 서명과 5바이트를 검사합니다. 실제 필요한 cycle은 약 100개(초기화 6 + 루프 10회 × 5 + 서명 저장·초기화 5 + UART 5회 × 7)이므로 여유가 큼니다.

7.4 실행과 기대 출력

```

$ make sim
iverilog -g2012 -Wall -o build/tb_core rtl/... tb/tb_rv32_core.v
vvp build/tb_core
PASS: RV32I smoke test

$ make sim-c-demo
... (c-demo 빌드) ...
bash scripts/run_c_demo_sim.sh
VCD info: dumpfile build/c_demo.vcd opened for output.
PASS: C firmware signature=55, UART="C OK\\n"

```

실패하면 FAIL: ... 메시지와 함께 \$fatal 이 종료 코드 1을 반환하고 make 가 오류로 끝납니다.

7.5 파형(VCD) 보는 법

```
gtkwave build/c_demo.vcd &
```

(VS Code에서는 WaveTrace 등 VCD 확장으로도 열 수 있습니다.) 추천 신호와 관찰 포인트:

신호	어디 서	무엇을 보나
clk, rst	tb	22 ns에 reset 해제
debug_pc	tb	매 edge마다 4씩 증가하다 분기에서 점프. 0x18~0x28 루프 10회, 0x40~0x58 루프 5회, 0x5c 고정
loop_i_x14, sum_x15	tb	i가 1..11, sum이 0,1,3,6,...,55로 변하는 모습
signature_write, signature_value	tb	PC=0x2c인 cycle에 1, 다음 edge에서 값이 55로
uart_valid, uart_data	tb	5개의 1-cycle pulse와 0x43,0x20,0x4F,0x4B,0x0A
dut.cpu.insn, opcode, rd_we, rd_data	cpu	디코더 출력
dut.cpu.dmem_addr, dmem_wstrb, dmem_wdata	cpu	sb 에서 wdata=0x43434343, wstrb=0001
dut.cpu.take_trap	cpu	이 프로그램에서는 항상 0 (trap 없음)

8. FPGA 구현 흐름과 결과 해석

8.1 전체 순서

```
make firmware # 1) HEX 최신화
make vivado # 2) 합성/구현/bitstream (수 분)
vivado -mode batch -source scripts/program_nexys_a7.tcl # 3) 보드에 다운로드
# 4) USB-UART 터미널 115200-8-N-1로 열기 (예: picocom -b 115200 /dev/ttyUSB1)
# 5) CPU_RESETN 버튼을 누르면 "Hello Nexys A7!"가 다시 출력됨
```

보드의 USB 하나로 JTAG(프로그래밍)과 UART(직렬 통신)가 함께 제공됩니다. Linux에서 보통 /dev/ttyUSB1 이 UART입니다.

8.2 실제 결과 (build/vivado/*.rpt)

항목	값	해석
WNS (Worst Negative Slack)	+3.850 ns	setup 여유. 양수이므로 모든 경로가 제 시간에 도착
WHS (Worst Hold Slack)	+0.139 ns	hold 여유. 양수이므로 데이터가 너무 빨리 바뀌는 경로 없음
Slice LUTs	4,614 / 63,400 (7.28%)	그중 LUT as Memory 2,048개가 8 KiB distributed RAM
Slice Registers	1,372 / 126,800 (1.08%)	레지스터 파일 992 + PC 32 + CSR + timer 128 + UART + reset sync
Block RAM Tile	0 / 135	BRAM을 전혀 쓰지 않음

왜 BRAM이 0인가: Artix-7 BRAM은 동기식(주소를 주면 다음 clock에 데이터)이지만 이 CPU는 같은 cycle에 명령어와 데이터를 모두 요구합니다(2.5절, 3.4절). 게다가 읽기 포트가 두 개(imem, dmem)에 바이트 단위 쓰기까지 있습니다. Vivado는 이런 배열을 LUT로 만든 distributed RAM으로 구현합니다. 8 KiB = 65,536비트이고 LUT 하나가 64비트 RAM이 되므로 최소 1,024개, 실제로는 두 읽기 포트 때문에 2,048개가 쓰였습니다. 메모리를 64 KiB로 키우면 LUT가 부족해집니다. BRAM을 쓰려면 CPU를 multi-cycle 또는 pipeline으로 바꿔 메모리 접근에 한 cycle을 주어야 합니다. README와 슬라이드가 "다음 단계"로 이것을 언급하는 이유입니다.

왜 12.5 MHz인가: single-cycle의 critical path(4.9절)가 80 ns 안에 들어옵니다(WNS +3.85 ns). 100 MHz(10 ns)에서는 불가능합니다. multi-cycle로 바꾸면 각 단계가 짧아져 clock을 올릴 수 있습니다.

8.3 리포트 파일

파일	내용
build/vivado/timing_summary.rpt	clock별 WNS/TNS/WHS/THS, 위반 경로 목록
build/vivado/utilization_synth.rpt	합성 직후 자원 사용량
build/vivado/utilization_route.rpt	배선 후 최종 자원 사용량
build/vivado/post_synth.dcp, post_route.dcp	Vivado GUI에서 open_checkpoint 로 열어 schematic/배치 확인
build/vivado/nexys_a7_rv32i.bit	보드에 쓸 bitstream

8.4 펌웨어를 바꿔 FPGA에 올리기

FPGA 이미지의 펌웨어는 `nexys_a7_top.v` 69행의 `MEM_HEX("firmware/nexys_hello.hex")` 로 정해집니다. `c_demo` 를 보드에서 돌리려면 이 문자열을 `"firmware/c_demo.hex"` 로 바꾸고 `make c-demo && make vivado` 를 실행합니다. HEX가 바뀔 때마다 **재합성이 필요합니다**(메모리 초기값이 bitstream에 구워지므로). 재합성 없이 프로그램을 바꾸려면 UART 수신(RX)과 부트로더가 필요한데, 현재는 RX가 없습니다.

9. Mini OS: 개념, 하드웨어 기반, 구현

Mini OS는 `firmware/boot.S`, `firmware/trap.S`, `firmware/kernel.c` 세 파일로 구현되어 있으며, `make sim-mini-os` 로 RTL 시뮬레이션에서 검증됩니다(9.7절에 실제 출력). 이 장은 (1) OS의 기본 개념, (2) 현재 하드웨어가 정확히 무엇을 제공하는지, (3) 그 위에서 이 코드가 어떻게 동작하는지를 설명합니다.

9.1 OS란 무엇인가 (이 프로젝트의 관점에서)

지금까지의 펌웨어(`nexys_hello`, `c_demo`)는 프로그램 하나가 CPU를 독점합니다. 반복문이 돌면 다른 일은 아무것도 못 합니다. OS의 가장 원초적인 역할은 여러 프로그램(task)이 CPU 하나를 나눠 쓰게 하는 것입니다. 이를 위해 필요한 최소 요소:

1. 강제로 제어권을 빼앗을 수단 — task가 협조하지 않아도 일정 시간마다 OS가 실행되어야 합니다. → **timer interrupt**
2. 실행 상태를 통째로 저장/복원하는 방법 — 중단된 task가 나중에 아무 일도 없었던 것처럼 이어서 실행되어야 합니다. → **context switch**
3. 다음에 누구를 실행할지 정하는 규칙 → **scheduler** (가장 단순한 것이 순서대로 돌리는 **round-robin**)
4. task가 OS에 서비스를 요청하는 통로 → **system call** (`ecall`)

"Mini"라는 말은 이 네 가지만, 그것도 가장 단순한 형태로 만든다는 뜻입니다. 메모리 보호, 파일 시스템, 가상 메모리, 사용자 모드는 없습니다(하드웨어에 U-mode와 MMU가 없으므로 애초에 불가능).

Context(문맥) 란 "task가 실행을 이어가는 데 필요한 모든 CPU 상태"입니다. 이 CPU에서는 `x1 ~ x31` 과 `PC` (정확히는 돌아갈 `PC = mepc`)가 전부입니다. 메모리는 task마다 각자의 스택을 쓰므로 저장할 필요가 없습니다.

9.2 현재 하드웨어가 제공하는 것

기능	위치	OS에서의 역할
주기적 timer interrupt	<code>simple_timer.v</code> , <code>rv32_csr.v</code> <code>irq_pending</code>	선점 (preemption) 신호
trap 진입 시 자동 동작: <code>mepc</code> , <code>mcause</code> 저장, <code>MIE←MPIE</code> , <code>MIE=0</code> , <code>PC=mtvec</code>	<code>rv32_csr.v</code> 98~102행, <code>rv32_core.v</code> 286행	handler 진입, 중첩 방지
trap 걸린 명령의 부작용 억제	<code>rv32_core.v</code> 102, 111, 120, 122행	정확한 재개 보장
<code>mret</code> : <code>PC=mepc</code> , <code>MIE←MPIE</code> , <code>MPIE=1</code>	<code>rv32_csr.v</code> 105~107행, <code>rv32_core.v</code> 250행	task로 복귀
<code>ecall</code> (cause 11)	<code>rv32_core.v</code> 248행	system call 진입
illegal instruction (cause 2)	<code>rv32_core.v</code> 276행	오류 검출
<code>mscratch</code>	<code>rv32_csr.v</code>	trap 진입 시 임시 저장소 (현재 구현은 사용하지 않음)
<code>mtvec</code> direct mode	<code>rv32_csr.v</code> 43행	handler 주소
<code>mip.MTIP</code> 읽기	<code>rv32_csr.v</code> 60행	pending 확인

기능	위치	OS에서의 역할
UART TX + ready	rv32_soc.v	task 출력

9.3 하드웨어에 없는 것 (구현이 우회하거나 감수한 것)

없는 것	영향	이 구현의 대응
하드웨어 스택 초기화	함수 호출 불가	boot.S 가 sp 를 _stack_top 으로 설정
reset 시 RAM 초기화	.bss 에 이전 실행의 값이 남음	boot.S 가 .bss 를 0으로 채움
mtime / mtimecmp 상위 32비트 MMIO	약 343초 후 IRQ 영구 on (4.5절)	데모는 그 전에 reset; RTL 개선 과제 (9.9절)
U-mode, 메모리 보호	task가 OS나 서로를 망가뜨릴 수 있음	협조적 task만 실행 (교육용 한계)
하드웨어 곱셈/나눗셈 (M 확장)	%, / 가 libgcc 함수 호출이 됨	-nostdlib 이므로 schedule() 은 % 대신 if (++cur >= NTASK) 사용
UART RX	입력 불가	출력 전용 데모
misaligned/access-fault trap, ebreak, wfi	잘못된 접근이 조용히 지나감	컴파일러 옵션과 코드 리뷰로 방지
timer 외 interrupt 소스	UART 등은 polling만 가능	uart_putc 가 ready를 polling

9.4 Trap 처리 흐름을 cycle 단위로

timer interrupt가 발생해서 돌아오기까지, 하드웨어와 소프트웨어가 번갈아 하는 일입니다.

```

cycle N    : mtime >= mtimecmp 성립 → timer_irq = 1 (level)
             mstatus.MIE=1, mie.MTIE=1 이면 irq_pending = 1 → take_trap = 1
             이 cycle의 명령어(PC=X)는 무효: rd_we/csr_we/dmem_wstrb 모두 0
             edge: pc <= mtvec, mepc <= X, mcause <= 0x80000007, MPIE <= 1, MIE <= 0
cycle N+1  : trap_entry 첫 명령 실행 (MIE=0이므로 timer_irq가 1이어도 다시 trap 안 걸림)
             ... : [trap.S] 레지스터 31개 + mepc 저장 → [kernel.c] mtimecmp 갱신(timer_irq = 0으로)
             → schedule() → [trap.S] sp 교체, 다른 task의 레지스터 복원
cycle M    : mret 실행. edge: pc <= mepc(=다음 task의 재개 주소), MIE <= MPIE(=1), MPIE <= 1
cycle M+1  : 다음 task의 명령 실행. 이 명령이 X'라면 X'는 지난번에 "무효 처리"됐던 바로 그 명령

```

핵심 규칙 세 가지 (모두 kernel.c에 반영):

1. handler는 반드시 mtimecmp를 미래로 옮긴 뒤에 mret 해야 합니다. 그렇지 않으면 mret 직후 MIE=1이 되는 순간 timer_irq가 아직 1이라 즉시 다시 trap이 걸려 task가 한 명령도 진행하지 못합니다. → timer_rearm()
2. ecall로 들어왔다면 handler가 mepc += 4를 해야 합니다. interrupt와 달리 ecall은 "실행된 것"으로 처리해야 하기 때문입니다(2.7절). → f->mepc += 4
3. handler 안에서는 interrupt가 꺼져 있으므로 handler를 짧게 유지해야 합니다. 이 하드웨어는 중첩 interrupt를 지원할 만한 상태(MPP 등)가 없습니다.

9.5 구현 개요

파일	역할	크기(FPGA 이미지 기준)
firmware/boot.S	reset 진입: sp 설정, .bss 초기화, mtvec 등록, kernel_main 호출	0x00~0x3F
firmware/trap.S	trap_entry: 문맥 저장 → trap_handler 호출 → 문맥 복원 → mret	0x40~0x1BF
firmware/kernel.c	timer 재설정, round-robin scheduler, ecall syscall, task 생성, task A/B/C, kernel_main	0x1C0~0x51F
firmware/linker.ld	_bss_start / _bss_end / _stack_top 심볼 추가	—
.bss	task_sp[4], stacks[3][256] (task당 1 KiB)	0x520~0x113F
커널 스택	_stack_top = 0x2000 에서 아래로	—

전체 4,409바이트(text 1,305 + bss 3,104)로 8 KiB에 들어갑니다. 위 주소는 build/firmware/mini_os.map 과 riscv64-unknown-elf-nm -n build/firmware/mini_os.elf 로 확인한 값입니다.

메모리 배치

```

0x0000 | _start (boot.S)
0x0040 | trap_entry (trap.S)
0x01C0 | kernel.c 코드/rodata
0x0520 | .bss: task_sp[4] | _bss_start
0x0540 | .bss: stacks[0] (task A, 1 KiB) ← 초기 프레임은 0x0940-128
0x0940 | .bss: stacks[1] (task B)
0x0D40 | .bss: stacks[2] (task C)
0x1140 | | _bss_end
      | (빈 공간)
      | 커널(idle) 스택 ↓
0x2000 | _stack_top

```

task 슬롯: 0 = 커널 idle 루프(kernel_main 끝의 for(;;) , 커널 스택 사용), 1 = task A, 2 = task B, 3 = task C.

9.6 코드 상세

9.6.1 firmware/linker.ld 추가분

```

.bss (NOLOAD) : ALIGN(4)
{
    _bss_start = .;
    *(.bss .bss.*)
    *(COMMON)
    . = ALIGN(4);
    _bss_end = .;
} > RAM

/* Kernel stack grows down from the top of RAM. Task stacks live in .bss. */
_stack_top = ORIGIN(RAM) + LENGTH(RAM);

```

_bss_start / _bss_end 는 boot 코드가 .bss 를 0으로 채우는 데 씁니다. 4.6.1절에서 본 대로 reset 버튼은 RAM을 초기화하지 않으므로, 두 번째 부팅부터는 이전 실행이 남긴 값이 .bss 에 남아 있습니다. 반드시 소프트웨어로 지워야

합니다. 기존 `c_demo/nexys_hello`는 이 심볼을 참조하지 않으므로 영향이 없습니다(`c_demo.map`의 배치가 변경 전과 동일함을 확인).

9.6.2 firmware/boot.S — 시작 코드

```
.option norvc
.section .text.start, "ax", @progbits
.globl _start
_start:
    la    sp, _stack_top           # 1) 스택 포인터. 이것이 있어야 C 함수 호출 가능

    la    t0, _bss_start          # 2) .bss 0으로 채우기
    la    t1, _bss_end

1:
    bgeu  t0, t1, 2f
    sw    zero, 0(t0)
    addi  t0, t0, 4
    j     1b

2:
    la    t0, trap_entry          # 3) trap handler 등록 (4바이트 정렬 필수)
    csrw  mtvec, t0

    call  kernel_main             # 4) C 커널로. task 생성, timer 설정, interrupt enable

3:
    j     3b                      # 돌아오면 안 되지만 안전망
```

`la` (load address)는 `auipc + addi`로 확장되는 pseudo-instruction이며 RV32I만으로 됩니다. `csrw`는 Zicsr 명령 (`csrrw x0, mtvec, t0`)입니다. 이 시스템의 GCC 9.3 / binutils 2.34는 `-march=rv32i`에서 CSR 명령을 받아들이지만, binutils 2.38 이후 toolchain에서는 `-march=rv32i_zicsr`이 필요합니다. 오류 `unrecognized opcode cswr`가 나면 이것이 원인입니다.

9.6.3 firmware/trap.S — trap 진입/복귀와 context 저장

context를 중단된 task의 스택 위에 128바이트 프레임으로 저장합니다. 프레임 배치: offset 0 = `mepc`, offset $4 \cdot n$ = `x_n` ($n=1..31$).

```
.section .text.trap, "ax", @progbits
.align 2                                # 2^2 = 4바이트 정렬. mtvec[1:0]=00 이어야 함
.globl trap_entry
trap_entry:
    addi  sp, sp, -128                # 프레임 확보
    sw    x1, 4(sp)
    sw    x3, 12(sp)                  # x2(sp)는 아래에서 별도 기록
    sw    x4, 16(sp)
    ...                               # x5..x30
    sw    x31, 124(sp)
    addi  t0, sp, 128                 # 원래 sp (디버깅 참고용)
    sw    t0, 8(sp)
    csrr  t0, mepc                    # 돌아갈 PC
    sw    t0, 0(sp)

    mv    a0, sp                      # a0 = 현재 프레임 포인터
    call  trap_handler                # C. 반환값 a0 = 다음에 실행할 task의 프레임 포인터
    mv    sp, a0                      # ★ 여기서 스택이 다른 task의 것으로 바뀜 = context switch

    lw    t0, 0(sp)
    csrw  mepc, t0
    lw    x1, 4(sp)
    lw    x3, 12(sp)
    ...                               # x4..x30
    lw    x31, 124(sp)
```

```

addi sp, sp, 128      # 프레임 해제 → sp가 그 task의 원래 sp
mret                  # PC ← mepc, MIE ← MPIE

```

설계 포인트:

- `t0` 을 쓰기 전에 모든 레지스터를 먼저 저장합니다(`t0` =x5도 20(sp)에 이미 저장됨).
- `call trap_handler` 는 `ra` (x1)를 덮어쓰지만 이미 저장했으므로 안전합니다.
- **context switch** 자체는 `mv sp, a0` 한 줄입니다. 저장은 "이 task의 스택"에, 복원은 "저 task의 스택"에서 하므로, 그 사이에 `sp` 만 바꾸면 됩니다. `trap_handler` 가 받은 프레임을 그대로 돌려주면 같은 task로 돌아갑니다.
- handler는 중단된 task의 스택에서 실행되므로 각 task 스택(1 KiB)에 프레임(128 B) + C handler가 쓰는 만큼의 여유가 있어야 합니다. 현재 handler의 스택 사용량은 수십 바이트입니다.
- 128은 16의 배수이므로 RISC-V ABI의 16바이트 스택 정렬이 유지됩니다.

9.6.4 firmware/kernel.c — scheduler, timer, syscall

프레임 구조체와 syscall 번호

```

struct frame {
    uint32_t mepc;
    uint32_t x[31];          /* x[i] holds register x(i+1) */
};
#define REG(f, n) ((f)->x[(n) - 1]) /* REG(f, 10) is a0, REG(f, 17) is a7 */

enum { SYS_PUTC = 1, SYS_YIELD = 2, SYS_GETTICK = 3 };

static struct frame *task_sp[NTASK];          /* NTASK = 4 */
static uint32_t stacks[NTASK - 1][STACK_WORDS] __attribute__((aligned(16)));
static int cur;
static volatile uint32_t ticks;

```

`struct frame` 은 `trap.S` 의 128바이트 배치와 정확히 같아야 합니다. `idle`(슬롯 0)은 커널 스택을 쓰므로 `stacks` 는 3개만 있습니다.

timer 재설정과 scheduler

```

static void timer_rearm(void)
{
    /* Clears the level IRQ and schedules the next tick in one write. */
    MTIMECMP_LO = MTIME_LO + TICK_CYCLES;
}

static struct frame *schedule(struct frame *f)
{
    task_sp[cur] = f;          /* remember where this task's frame is */
    if (++cur >= NTASK)        /* no '%' : RV32I has no divider and */
        cur = 0;              /* libgcc is not linked */
    return task_sp[cur];
}

```

`schedule()` 은 "현재 프레임 위치를 기억하고 다음 슬롯의 프레임을 돌려준다"가 전부입니다. round-robin이므로 우선순위도 대기 상태도 없습니다.

trap handler

```

struct frame *trap_handler(struct frame *f)
{
    uint32_t cause = csr_read_mcause();

    if (cause == CAUSE_MTIMER) {                /* 0x80000007 */
        ticks++;
        timer_rearm();                          /* before mret, or the IRQ re-fires */
        return schedule(f);
    }

    if (cause == CAUSE_ECALL_M) {                /* 11 */
        f->mepc += 4;                            /* resume after the ecall itself */
        switch (REG(f, 17)) {                    /* a7 = system call number */
            case SYS_PUTC:    uart_putc((char)REG(f, 10)); return f;
            case SYS_YIELD:   return schedule(f);
            case SYS_GETTICK: REG(f, 10) = ticks; return f;
            default:          REG(f, 10) = (uint32_t)-1; return f;
        }
    }

    panic(cause, f->mepc);                       /* illegal instruction or unknown */
    return f;
}

```

syscall 인자는 프레임에 저장된 a0 (REG(f,10))에서 읽고, 반환값도 프레임의 a0 자리에 써 두면 복원 시 task의 a0 가 됩니다. 알 수 없는 원인(illegal instruction 등)이면 panic() 이 UART에 PANIC cause=... pc=... 를 출력하고 멈춥니다.

task 생성

```

static void task_create(int slot, void (*entry)(void))
{
    uint32_t *top = &stacks[slot - 1][STACK_WORDS];
    struct frame *f = (struct frame *)((uint8_t *)top - sizeof(struct frame));
    int i;

    for (i = 0; i < 31; i++)
        f->x[i] = 0;
    f->mepc = (uint32_t)entry;
    REG(f, 1) = (uint32_t)task_exit; /* ra */
    REG(f, 2) = (uint32_t)top;       /* sp slot, informational */
    task_sp[slot] = f;
}

```

핵심 아이디어: 새 task를 "처음 시작"하는 특별한 경로가 없습니다. 마치 방금 interrupt로 중단된 것처럼 보이는 가짜 프레임을 스택 꼭대기에 만들어 두면, trap.S의 일반 복원 경로가 mret 로 entry 에 "돌아가" 줍니다. mret 시 MIE ← MPIE 인데, trap 진입 때 MPIE ← MIE(=1)이었으므로 새 task는 interrupt가 켜진 채 시작됩니다.

syscall 스텝과 task

```

static void sys_putc(char c)
{
    register uint32_t a0 __asm__("a0") = (uint8_t)c;
    register uint32_t a7 __asm__("a7") = SYS_PUTC;
    __asm__ volatile ("ecall" : "+r"(a0) : "r"(a7) : "memory");
}

static void task_a(void) { for (;;) sys_putc('A'); }
static void task_b(void) { for (;;) sys_putc('B'); }
static void task_c(void) { for (;;) { sys_putc('C'); sys_yield(); } }

```

register ... __asm__("a0") 는 GCC에게 그 변수를 특정 레지스터에 두라는 지시입니다. A와 B는 시간 슬라이스가 끝날 때까지 계속 출력하고(선점 확인), C는 한 글자 출력 후 SYS_YIELD 로 자발적으로 CPU를 넘깁니다(협조적 전환 확인).

kernel_main

```
void kernel_main(void)
{
    task_create(1, task_a);
    task_create(2, task_b);
    task_create(3, task_c);
    cur = 0;                                /* we are the idle task, slot 0 */

    uart_puts("mini OS boot\n");

    timer_rearm();
    csr_write_mie(0x80u);                   /* MTIE */
    csr_set_mstatus_mie();                  /* MIE: from here on we can be preempted */

    for (;;) { }                           /* idle */
}
```

동작 순서:

1. _start → kernel_main: task 1~3의 초기 프레임을 각자의 스택 꼭대기에 만들고, 배너를 출력한 뒤, timer와 interrupt를 켜고 idle 루프에 들어갑니다.
2. 첫 timer trap. trap_entry 가 idle 문맥을 커널 스택에 저장 → trap_handler 가 task_sp[0] 에 기록하고 task_sp[1] 을 반환 → trap_entry 가 그 프레임을 복원하고 mret. mepc = task_a 이므로 task A가 처음으로 실행됩니다.
3. task A는 ecall 로 문자를 출력합니다. handler는 mepc += 4, UART 쓰기, 같은 프레임 반환 → A로 복귀.
4. 다음 timer trap에서 A의 문맥이 A의 스택에 저장되고 B로 전환. B 슬라이스 후 C로 전환. C는 한 글자 쓰고 yield → schedule() → 슬롯 0(idle). idle은 다음 timer까지 돕니다. 이후 A → B → C → idle → ... 순환.

9.7 빌드와 시뮬레이션 결과

Makefile 타겟 (MINI_OS_* 변수, 6.2절 형식과 동일)

명령	생성물
make mini-os	firmware/mini_os.hex (FPGA용, TICK_CYCLES=125000 = 12.5 MHz에서 10 ms), build/firmware/mini_os.{elf,bin,dis,map}
make mini-os-disasm	역어셈블 출력
make sim-mini-os	build/firmware/mini_os_sim.hex (TICK_CYCLES=2000), build/tb_mini_os, build/mini_os.vcd, PASS/FAIL

같은 소스에서 -DTICK_CYCLES= 만 다른 두 이미지를 만듭니다. 시뮬레이션 이미지의 짧은 tick은 짧은 실행에서 많은 전환을 보기 위한 것입니다. MINI_OS_TICK, MINI_OS_SIM_TICK 으로 바꿀 수 있습니다.

테스트벤치 tb/tb_mini_os.v: rv32_soc 에 시뮬레이션 이미지를 적재하고 80,000 cycle 실행하면서 UART 바이트와 모든 trap(dut.cpu.take_trap, dut.cpu.trap_cause)을 기록한 뒤 검사합니다: 배너 "mini OS

boot\n" 이 먼저 나왔는가, timer trap ≥ 8 , 예상 밖 trap(illegal 등) = 0, A/B/C 모두 출력했는가, UART에서 관찰된 task 전환 ≥ 6 .

실제 실행 결과 (make sim-mini-os):

```
--- UART output (431 bytes, first 300 shown) ---
mini OS boot
AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBCAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBBCAAAAAAAAAAAAAAAABBBBBBBBB
BBBBBBBBBC...
--- end of UART output ---
timer traps=35  ecall traps=430  other traps=0
A=204  B=203  C=11  task switches seen on UART=34
PASS: mini OS boot, 35 timer IRQs, 430 syscalls, tasks A/B/C interleaved
```

읽는 법:

- 2,000 cycle 슬라이스 동안 A가 17글자를 출력했습니다. 즉 ecall 왕복(trap 진입 + 저장 31개 + handler + 복원 31개 + mret) 하나가 약 115 cycle입니다.
- C가 한 글자씩만 보이는 것은 yield 때문입니다. C 다음에 idle 슬라이스(출력 없음)가 오고 다시 A입니다.
- $80,000 \text{ cycle} \div 2,000 = 40 \text{ tick}$ 중 35회가 trap으로 기록된 것은 부팅 구간과 마지막 미완 슬라이스 때문입니다.
- other traps=0 이 가장 중요한 검사입니다. 컴파일러가 RV32I 밖의 명령을 냈거나 프레임 배치가 어긋났다면 여기서 illegal instruction이 잡힙니다.

파형 (gtkwave build/mini_os.vcd): sp_x2가 0x1Fxx (커널) → 0x08xx (A) → 0x0Cxx (B) → 0x10xx (C) 사이클 오가는 것, take_trap과 trap_cause, mie_bit (handler 동안 0), timer_irq (handler가 mtimecmp를 쓰면 즉시 0)를 보면 9.4절의 cycle 흐름을 그대로 확인할 수 있습니다.

9.8 FPGA에서 실행하기

rtl/nexys_a7_top.v의 MEM_HEX("firmware/nexys_hello.hex")를 "firmware/mini_os.hex"로 바꾸고:

```
make mini-os
make vivado
vivado -mode batch -source scripts/program_nexys_a7.tcl
```

UART 터미널(115200-8-N-1)에 mini OS boot 다음 10 ms마다 task가 바뀌며 A/B/C가 나옵니다. 115200 baud에서 10 ms는 약 115글자이므로 A와 B 구간이 눈에 띄게 길다. 전환을 육안으로 보려면 make mini-os MINI_OS_TICK=12500000 (1초)으로 빌드하십시오. 약 5분 43초 후에는 timer 상위 word 문제(4.5절)로 IRQ가 영구히 켜져 handler만 반복 실행되므로 그 전에 CPU_RESETN을 누르십시오(reset은 mtime을 0으로 되돌립니다).

9.9 다음 단계

하드웨어

1. timer 상위 32비트 MMIO (0x1000_1008, 0x1000_100C): simple_timer.v에 addr[3:2] 디코딩을 추가하면 됩니다. 이후 표준 64-bit mtimecmp 쓰기 순서(상위를 먼저 최대값으로, 하위, 상위 순)를 소프트웨어가 따릅니다.
2. UART RX: 입력을 받으면 셀 같은 대화형 데모가 가능해지고, 부트로더로 재합성 없이 프로그램을 바꿀 수 있습니다.
3. software interrupt(msip)와 mie.MSIE: task 간 신호나 "즉시 스케줄"에 유용합니다.

4. `misaligned/access-fault trap`과 `ebreak` : 디버깅이 쉬워집니다.

5. `multi-cycle` 또는 `pipeline`: BRAM 사용과 clock 상승. 메모리를 64 KiB 이상으로 키우려면 필요합니다.

소프트웨어

1. task 상태(`ready/blocked`)와 `SYS_SLEEP(ticks)` : `ticks` 를 이용해 일정 시간 대기하는 task.

2. 간단한 mutex 또는 UART 출력 버퍼: 지금은 task가 슬라이스 중간에 선점되면 다른 task의 글자와 섞일 수 있습니다 (ecall 단위로는 원자적).

3. `mscratch` 를 이용한 커널 전용 trap 스택: task 스택이 넘쳐도 handler가 살아남게 합니다.

10. 자주 하는 질문과 문제 해결

Q. `make sim-c-demo`가 "Icarus Verilog is required"로 실패합니다. `sudo apt install iverilog`로 설치하거나, 관리자 권한이 없으면 `.deb`를 `dpkg -x`로 `build/tools/iverilog/`에 풀어 두면 스크립트가 자동으로 찾습니다(6.7절).

Q. `$readmemh` 경고 "file not found"가 나옵니다. HEX 경로가 실행 디렉토리 기준 상대 경로입니다. 프로젝트 루트에서 실행하십시오. `make`와 스크립트는 자동으로 루트에서 실행합니다.

Q. 컴파일한 C 프로그램이 CPU에서 이상하게 동작합니다. 순서대로 확인: (1) `.dis`에 `mul`, `c`, 접두 명령, `amo*`가 없는가 (`-march=rv32i` 누락), (2) 스택을 쓰는 코드인데 `sp`를 초기화했는가 (`-00`이나 함수 호출이 있으면 필요), (3) `gp` 상대 접근이 있는가 (`-msmall-data-limit=0`, `-mno-relax` 누락), (4) 전역 변수가 `.bss`에 있는데 `reset` 후 0이라고 가정했는가.

Q. unrecognized opcode csrw 오류가 납니다. 새 binutils입니다. `-march=rv32i_zicsr`을 쓰십시오(9.6.2절).

Q. 프로그램이 8 KiB를 넘습니다. 링커가 `region RAM overflowed`, 또는 `bin2hex.py`가 `image is N bytes, larger than 8192-byte memory`를 냅니다. 코드를 줄이거나, `nexys_a7_top.v`의 `MEM_WORDS`, `linker.ld`의 `LENGTH`, `Makefile`의 `--max-bytes`를 함께 키우십시오(LUT 사용량이 비례해 증가합니다, 8.2절).

Q. Vivado가 보드를 찾지 못합니다. `program_nexys_a7.tcl`이 `xc7a100t_0 not found`를 냅니다. USB 케이블(JTAG 포트), 보드 전원 스위치, Digilent 케이블 드라이버(`install_digilent.sh`)를 확인하십시오.

Q. UART 터미널에 아무것도 안 나옵니다. (1) 포트가 `/dev/ttyUSB1` (보통 두 번째)인지, (2) 115200-8-N-1인지, (3) `CPU_RESETN`을 눌러 프로그램을 재시작했는지(출력은 부팅 시 한 번만), (4) LED에 PC가 표시되는지(9 = 0b1001 이면 정상 종료 루프) 확인하십시오.

Q. timer interrupt가 걸리지 않습니다. `mstatus.MIE` (bit 3)와 `mie.MTIE` (bit 7)를 둘 다 켜는지, `mtimecmp`를 reset 기본값 `0xFFFFFFFF`에서 바꿨는지 확인하십시오(4.3.2절, 4.5절).

Q. handler에서 돌아오자마자 다시 trap이 걸립니다. timer라면 `mtimecmp`를 미래로 옮기지 않은 것, `ecall`이라면 `mepc += 4`를 안 한 것입니다(9.4절).

Q. `make lint`가 경고를 냅니다. `-Wall`은 스타일 경고까지 포함합니다. 미사용 신호(예: `trap_taken open`) 경고는 의도된 것입니다. `%Error`가 아니면 진행에 문제없습니다.

Q. `c_demo.vcd`가 루트와 `build/` 두 곳에 있습니다. 테스트벤치는 `build/c_demo.vcd`에 씁니다. 루트의 것은 이전에 다른 방식으로 실행했을 때 남은 사본입니다.

부록 A. RV32I 명령어 인코딩 표

이 CPU가 디코딩하는 모든 명령어입니다. (opcode는 `insn[6:0]`, funct3은 `[14:12]`, funct7은 `[31:25]`)

명령어	형식	opcode	funct3	funct7/imm[11:5]	동작
<code>lui</code>	U	0110111	—	—	$rd = \text{imm} \ll 12$
<code>auipc</code>	U	0010111	—	—	$rd = pc + (\text{imm} \ll 12)$
<code>jal</code>	J	1101111	—	—	$rd = pc+4; pc += \text{imm}$
<code>jalr</code>	I	1100111	000	—	$rd = pc+4; pc = (rs1 + \text{imm}) \& \sim 1$
<code>beq</code>	B	1100011	000	—	if $rs1 == rs2$: $pc += \text{imm}$
<code>bne</code>	B	1100011	001	—	if $rs1 \neq rs2$
<code>blt</code>	B	1100011	100	—	if $rs1 < rs2$ (signed)
<code>bge</code>	B	1100011	101	—	if $rs1 \geq rs2$ (signed)
<code>bltu</code>	B	1100011	110	—	if $rs1 < rs2$ (unsigned)
<code>bgeu</code>	B	1100011	111	—	if $rs1 \geq rs2$ (unsigned)
<code>lb</code>	I	0000011	000	—	$rd = \text{sext}(\text{mem8}[rs1 + \text{imm}])$
<code>lh</code>	I	0000011	001	—	$rd = \text{sext}(\text{mem16})$
<code>lw</code>	I	0000011	010	—	$rd = \text{mem32}$
<code>lbu</code>	I	0000011	100	—	$rd = \text{zext}(\text{mem8})$
<code>lhu</code>	I	0000011	101	—	$rd = \text{zext}(\text{mem16})$
<code>sb</code>	S	0100011	000	—	$\text{mem8}[rs1 + \text{imm}] = rs2[7:0]$
<code>sh</code>	S	0100011	001	—	$\text{mem16} = rs2[15:0]$
<code>sw</code>	S	0100011	010	—	$\text{mem32} = rs2$
<code>addi</code>	I	0010011	000	—	$rd = rs1 + \text{imm}$
<code>slti</code>	I	0010011	010	—	$rd = (rs1 < \text{imm}) \text{ signed}$
<code>sltiu</code>	I	0010011	011	—	$rd = (rs1 < \text{imm}) \text{ unsigned}$
<code>xori</code>	I	0010011	100	—	$rd = rs1 \wedge \text{imm}$
<code>ori</code>	I	0010011	110	—	$rd = rs1 \mid \text{imm}$
<code>andi</code>	I	0010011	111	—	$rd = rs1 \& \text{imm}$
<code>slli</code>	I	0010011	001	0000000	$rd = rs1 \ll \text{shamt}$
<code>srl</code>	I	0010011	101	0000000	$rd = rs1 \gg \text{shamt}$ (논리)
<code>srai</code>	I	0010011	101	0100000	$rd = rs1 \gg \text{shamt}$ (산술)
<code>add</code>	R	0110011	000	0000000	$rd = rs1 + rs2$
<code>sub</code>	R	0110011	000	0100000	$rd = rs1 - rs2$
<code>sll</code>	R	0110011	001	0000000	$rd = rs1 \ll rs2[4:0]$
<code>slt</code>	R	0110011	010	0000000	signed 비교

명령	형식	opcode	funct3	funct7/imm[11:5]	동작
sltu	R	0110011	011	0000000	unsigned 비교
xor	R	0110011	100	0000000	
srl	R	0110011	101	0000000	
sra	R	0110011	101	0100000	
or	R	0110011	110	0000000	
and	R	0110011	111	0000000	
fence	I	0001111	000	—	NOP 취급
ecall	I	1110011	000	imm=0	trap, cause 11
mret	I	1110011	000	insn=0x30200073	pc = mepc, MIE = MPIE
csrrw	I	1110011	001	csr	rd = csr; csr = rs1
csrrs	I	1110011	010	csr	rd = csr; csr = rs1 (rs1 ≠ x0일 때)
csrrc	I	1110011	011	csr	rd = csr; csr &= ~rs1 (rs1 ≠ x0일 때)
csrrwi	I	1110011	101	csr	csr = zimm
csrrsi	I	1110011	110	csr	csr = zimm (zimm ≠ 0일 때)
csrrci	I	1110011	111	csr	csr &= ~zimm (zimm ≠ 0일 때)

자주 쓰는 pseudo-instruction: `nop = addi x0,x0,0`, `li rd,imm = addi / lui+addi`, `mv rd,rs = addi rd,rs,0`, `j off = jal x0,off`, `call f = auipc ra+jalr ra`, `ret = jalr x0,0(ra)`, `beqz rs = beq rs,x0`, `bnez rs = bne rs,x0`, `csrr rd,csr = csrrs rd,csr,x0`, `csrw csr,rs = csrrw x0,csr,rs`, `csrrsi csr,imm = csrrsi x0,csr,imm`.

부록 B. 구현된 CSR 표

주소	이름	구현된 비트	읽기	쓰기	trap 진입 시	mret 시
0x300	mstatus	MIE[3], MPIE[7]	저장값	두 비트만	MPIE←MIE, MIE←0	MIE←MPIE, MPIE←1
0x304	mie	MTIE[7]	저장값	bit 7만	—	—
0x305	mtvec	[31:2]	저장값	하위 2비트 0 강제	—	—
0x340	mscratch	전체	저장값	전체	—	—
0x341	mepc	[31:2]	저장값	하위 2비트 0 강제	←trap PC	—
0x342	mcause	전체	저장값	전체	←cause	—
0x344	mip	MTIP[7]	timer_irq 반영	무시	—	—
0xF11~0x F14	mvendorid/marchid/mimp id/mhartid	—	0	무시	—	—
그 외	—	—	illegal instruction	illegal instruction	—	—

cause 값: 2 illegal instruction, 11 ecall from M-mode, 0x8000_0007 machine timer interrupt.

부록 C. 손으로 명령어 디코딩하기

tb_rv32_core.v의 32'h08302023을 예로 듭니다.

```
0x08302023 = 0000 1000 0011 0000 0010 0000 0010 0011
비트 31..25 = 0000100 → imm[11:5]
비트 24..20 = 00011 → rs2 = x3
비트 19..15 = 00000 → rs1 = x0
비트 14..12 = 010 → funct3 = SW
비트 11..7 = 00000 → imm[4:0]
비트 6..0 = 0100011 → opcode = STORE
imm = {imm[11:5], imm[4:0]} = 0000100_00000 = 0x080 = 128
→ sw x3, 128(x0)
```

rv32_core.v가 하는 일도 정확히 이것입니다: opcode로 STORE 분기 진입, `imm_s(insn) = 128`, `daddr_r = rs1_data(0) + 128`, `funct3=010`이므로 `dwstrb_r = 1111`, `dwddata_r = rs2_data`.

분기 offset 예: 0xfe020ee3 (beq x4,x0,-4)

```
1111 1110 0000 0010 0000 1110 1110 0011
bit31=1 (imm[12], 부호), bits30:25=111111 (imm[10:5]), bits11:8=1110 (imm[4:1]), bit7=1
(imm[11])
imm = {imm[12], imm[11], imm[10:5], imm[4:1], 0} = 1_1_11111_1110_0 = 0b1111111111100 (13비트)
= -4
```

부록 D. 용어집

용어	설명
ABI	Application Binary Interface. 레지스터 용도, 호출 규약, 데이터 크기의 약속
ALU	Arithmetic Logic Unit. 덧셈·비교·논리·시프트를 하는 조합 회로
Baud	초당 심볼 수. UART에서는 초당 비트 수
BRAM / distributed RAM	FPGA의 전용 메모리 블록 / LUT로 만든 작은 메모리
BUFG	Xilinx global clock buffer
CDC	Clock Domain Crossing. 서로 다른 clock 영역 사이의 신호 전달
Context switch	실행 중인 task의 CPU 상태를 저장하고 다른 task의 상태를 복원하는 것
CSR	Control and Status Register. 특권 상태를 담는 레지스터군
Critical path	조합 논리 중 가장 긴 지연 경로. clock 주기의 하한을 정함
ECALL / MRET	소프트웨어 trap 요청 / trap에서 복귀
ELF	실행 파일 형식. 섹션·심볼·주소 정보 포함
Freestanding	OS와 표준 라이브러리가 없는 C 실행 환경
Harvard / unified	명령어와 데이터 메모리 분리 / 통합. 이 SoC는 인터페이스는 분리, 저장소는 통합
HEX (\$readmemh)	한 줄에 한 word씩 16진수로 적은 메모리 초기화 텍스트
Hold / Setup slack	데이터가 clock edge 이후 충분히 유지되는가 / 이전에 충분히 도착하는가의 여유
Immediate	명령어에 내장된 상수
Interrupt / Exception	외부 원인의 비동기 trap / 명령어 자체가 원인인 동기 trap
Latch	의도치 않게 생기는 레벨 감지 기억 소자. 조합 블록에서 대입 누락 시 발생
Level / Pulse	조건이 성립하는 동안 유지되는 신호 / 한 cycle만 1인 신호
Linker script	코드/데이터의 메모리 배치 규칙
Little-endian	최하위 바이트를 낮은 주소에 저장
LUT	Look-Up Table. FPGA의 기본 논리 소자
M-mode	Machine mode. 최고 특권 모드
MMIO	Memory-Mapped I/O. 주변장치를 메모리 주소로 접근
mtime / mtimecmp	자유 실행 카운터 / 비교값. <code>mtime >= mtimecmp</code> 이면 timer interrupt
Non-blocking (<=)	순차 논리에서 edge 시점의 값으로 동시에 갱신되는 대입
PC	Program Counter. 현재 명령어 주소
Preemption	task의 동의 없이 OS가 CPU를 회수하는 것
Relaxation	링커가 주소 접근 명령을 더 짧은 형태로 바꾸는 최적화
Round-robin	task를 순서대로 같은 시간씩 실행하는 scheduler
RTL	Register Transfer Level. 합성 가능한 HDL 기술 수준
Single-cycle	모든 명령어가 한 clock cycle에 완료되는 구조

용어	설명
STA	Static Timing Analysis. 모든 경로의 지연을 계산해 timing 충족 여부를 판정
System call	사용자 코드가 OS 서비스를 요청하는 통로 (<code>ecall</code>)
Trap	정상 흐름을 중단하고 handler로 점프하는 사건의 총칭
Trap handler	trap 발생 시 실행되는 코드. <code>mtvec</code> 가 가리킴
UART	비동기 직렬 통신. start/data/stop 비트
VCD	Value Change Dump. 파형 파일 형식
WNS / WHS	Worst Negative Slack / Worst Hold Slack. 양수면 timing 충족
Write strobe / byte enable	word 중 어느 바이트를 쓸지 나타내는 비트들
XDC	Xilinx Design Constraints. 핀·clock 제약 파일
Zicsr	CSR 접근 명령 확장